

Bounded Memory Computation for Language Models: A Survey of Long Context, Retrieval, Native-KV Memory, and Progressive Access

Jorge Simão

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2026

Abstract

Language models can enlarge attention, carry recurrent state, retrieve text, query latent memory, retain or compress K/V, navigate an index, or call tools. These mechanisms are usually compared by context length or benchmark score. We organize them instead by the *memory computation* they implement: what is logically available; what becomes causally active; how candidates are discovered and searched; which detail is materialized; where it enters model depth; how it changes decisions and free-running behavior; and which serving resources it consumes. This decomposition reveals recurring category errors. Logical availability is not active computation, semantic retrieval geometry is not native attention geometry, discovery is not integration, and teacher-forced likelihood is not generated behavior.

The survey maps seven established families onto nine computational questions and a common comparison taxonomy. Progressive Retrieval Attention (PRAAttention, PRA) is a worked synthesis, not a presumed winner. Its tested core separates bounded logical memory from a smaller active working set, routes over compact gists, and transports selected pretrained native K/V without changing the disabled path. Results across standalone and pretrained backbones establish fragmentation repair, bounded execution, exact native transport, position-aware cache correctness, a learned semantic router, depth-sensitive integration, and one-shot QASPER benefit. They also expose limits: HotpotQA errors are often relation-near but evidence-incomplete, one-shot retrieval does not provide associative closure, and selected relevance does not imply that every token in a selected chunk is required. We therefore separate established PRA mechanisms from proposed iterative gist traversal and later smart materialization. The result is a conceptual and experimental map for bounded memory systems, including causal controls and quality-resource frontiers that apply beyond PRA.

Contents

1	Introduction	4
1.1	Seven Access Families	4
1.2	A Shared Vocabulary	4
1.3	Nine Questions for Any Memory Architecture	5
1.4	Experimentally Forced Distinctions	5
1.5	How to Read the Comparisons	5
2	Progressive Retrieval Attention	6
2.1	PRA as a Worked Decomposition	6
2.2	Identity, Resolution, and Bounded Logical Memory	6
2.3	Encoding Granularity, Routing Granularity, and Native Payload	6
2.4	Gists and Search Geometry	7
2.5	Position Is Not Context	7
2.6	Depth, Integration, Decision, and Behavior	7
2.7	Evidence Ledger	8
2.8	Systems Accounting	8
2.9	Causal Evaluation Doctrine	9
2.10	Architecture Ladder: Established, Next, and Later	9

3	Category I: Dense, Sparse, and Approximate Long-Context Attention	9
3.1	Dense attention and positional extension	10
3.2	Longformer	10
3.3	BigBird	11
3.4	Performer	11
4	Category II: Recurrence and Compression	12
4.1	Transformer-XL	12
4.2	Compressive Transformer	12
4.3	Recurrent Memory Transformer (RMT)	13
4.4	Block-Recurrent Transformer	13
4.5	TransformerFAM	14
4.6	Infini-attention	15
4.7	Titans and Test-Time Neural Memory	15
4.8	Selective State-Space Models	16
5	Category III: Differentiable and Episodic External Memory	16
5.1	Memory Networks	16
5.2	Differentiable Neural Computer (DNC)	17
5.3	Memformer	17
5.4	Memorizing Transformer	18
6	Category IV: Retrieval-Augmented Pretraining and Generation	18
6.1	kNN-LM	19
6.2	REALM	19
6.3	Retrieval-Augmented Generation (RAG)	20
6.4	Atlas	20
6.5	Self-RAG	21
6.6	Iterative Retrieval and Associative Closure	21
6.7	RETRO	21
7	Category V: Attention-Level Retrieval and KV Runtime Management	22
7.1	Unlimiformer	22
7.2	Landmark Attention	23
7.3	PagedAttention	23
7.4	StreamingLLM	24
7.5	H ₂ O: Heavy-Hitter Oracle	24
7.6	Scissorhands	25
7.7	SnapKV	25
7.8	FastGen and PyramidKV: Head- and Layer-Adaptive Budgets	26
7.9	Quest: Query-Aware Page Selection	26
7.10	MiniCache and CacheBlend: Depth Compression and Reusable Chunks	27
7.11	Prompt Cache and KV Quantization	27
7.12	KV management design map	27
8	Category VI: Structural and Hierarchical Indexing	28
8.1	Hierarchical Attention Networks (HAN)	28
8.2	RAPTOR	29
8.3	MemWalker	29
8.4	PageIndex	30

8.5	GraphRAG	30
8.6	PaperTrail as provenance and HCI	31
9	Category VII: Agents with Search and Tool Use	31
9.1	ReAct	31
9.2	Self-Ask with Search	32
9.3	IRCoT	32
9.4	Toolformer	33
9.5	WebGPT and browser agents	33
9.6	Recursive Language Models and context as environment	34
9.7	A tiered agent-PRA design	35
10	Cross-Cutting Conditional Computation	35
11	One-Page Architecture Landscape	36
12	Cross-Architecture Comparison	37
13	Adapting Original Experiments to PRA	38
13.1	A benchmark matrix	38
13.2	Common conditions	38
13.3	PRA-specific causal matrix	39
13.4	Metrics beyond task accuracy	39
14	Synthesis Across the Nine Questions	39
14.1	Availability and causal activity	39
14.2	Discovery, geometry, and closure	39
14.3	Detail, depth, and behavior	40
14.4	Serving and resource frontiers	40
15	Research Lineage and Reusable Related-Work Synthesis	40
16	Research Frontier	41
16.1	Multi-gist semantic resolution	41
16.2	Iterative associative closure and path preservation	41
16.3	Progressive and smart materialization	41
16.4	Production serving	41
17	Cognition as a Bounded Analogy	41
18	Discussion	41
19	Conclusion	42

1 Introduction

The canonical Transformer gives every token in a bounded sequence access to preceding tokens through causal self-attention. A longer sequence expands what is logically present, but it does not guarantee that the right information will be found, represented faithfully, used at the right layers, converted into a decision, or expressed in a completed answer. Conversely, a retrieval system may expose only a small working set while retaining a much larger logical store. “Context length” therefore collapses several different computational questions into one number.

Consider a model repairing a software repository. A dense-context system loads files eagerly. A sparse Transformer changes their interaction graph. A retriever inserts passages. A structural navigator follows repository, module, class, and method nodes. An agent searches, runs tests, and revises its plan. A recurrent or state-space model carries compressed history. A K/V system may retain, evict, quantize, page, or reuse hidden detail. These systems place the boundary between available information and active computation in different locations. Their failures also occur at different boundaries: source discovery, semantic ranking, detail transport, deep integration, decision formation, answer realization, or serving.

This survey treats that boundary as an *access contract*. The contract states what is stored, how it is named, how it is searched, what is materialized, where it enters the model, how it may be invalidated, and which intervention would show causal use. The seven method families remain a useful literature atlas. The nine questions below provide the stronger cross-family analysis. PRA then serves as a worked decomposition because it deliberately separates naming, routing, native-KV materialization, and layer-local consumption. Presenting it early is pedagogical, not an evidential ranking.

1.1 Seven Access Families

The seven categories below follow the location at which an architecture changes the information path. They are not mutually exclusive. A practical system may use sparse local attention, a retrieval index, a K/V cache, and an agent escalation policy at the same time.

1. **Dense, sparse, and approximate long-context attention:** extend the visible token graph.
2. **Recurrence and compression:** carry activations, latent slots, associative state, or test-time neural memory across segments.
3. **Differentiable external memory:** read and write learned memory slots or episodic stores.
4. **Retrieval-augmented pretraining and generation:** retrieve textual evidence before or during generation.
5. **Attention-level and KV retrieval:** select, retain, compress, allocate, or reuse hidden-state and key/value memories near attention.
6. **Structural and hierarchical indexing:** navigate document, graph, or summary structure rather than flat chunks.
7. **Agents with search tools:** iteratively formulate queries, inspect results, follow links, and decide when to stop.

1.2 A Shared Vocabulary

The repository example suggests a decomposition that applies across categories. First, a name or index entry says what information is available; this may be a URI, document identifier, token position, or search result. The model then has a local state, such as the hidden representation of the failing test. A policy chooses candidates, perhaps an authentication module or a prior incident. The system materializes those candidates as source tokens, summaries, hidden states, K/V tensors, graph nodes, or tool observations. Finally, a fusion mechanism makes the representation available to the model. A compact item used to choose evidence should not be confused with the detailed evidence supplied after that choice.

We can express this process without committing to a particular architecture. Let $x_{1:T}$ denote the local tokens and let h_t be the model state at generation step t . A query q_t is derived from that state. The external store is $\mathcal{S}$, and R is a selection policy with configuration π . The selected candidates are

$$C_t = R(q_t, \mathcal{S}; \pi), \tag{1}$$

$$M_t = \text{materialize}(C_t; \rho), \tag{2}$$

$$h'_t = \text{combine}(h_t, M_t; \gamma), \tag{3}$$

where C_t is the candidate set. The materialization policy ρ determines whether those candidates become tokens, latent states, K/V tensors, or observations. The fusion policy γ determines whether M_t is concatenated to a prompt, read by cross-attention, injected into a cache, or processed by another model call. This vocabulary separates three questions that are often conflated: whether the right information was selected, whether its representation preserved the needed evidence, and whether the model used that representation. The next section applies the decomposition to PRA before the survey compares it with the other six families.

1.3 Nine Questions for Any Memory Architecture

The same system can answer these questions differently, and success at one does not imply success at the next.

1. **Logical availability:** which source tokens, objects, states, or records can in principle be reached?
2. **Causal activity:** which of them participate in the current model operation?
3. **Discovery:** how does an unknown source enter the candidate set?
4. **Search geometry:** which representation and similarity rule rank candidates?
5. **Relevance through retrieved concepts:** can an initial hit reveal another necessary source, relation, or query?
6. **Materialized detail:** after selection, which text, hidden states, or native K/V are exposed, and at what granularity?
7. **Model depth:** at which layers is memory useful, ignored, or disruptive?
8. **Decisions and behavior:** does memory change answer-relevant logits and complete generated answers, rather than only average likelihood?
9. **Serving and resources:** what are the cold/warm latency, active K/V, backing-store bytes, transfer, synchronization, and reuse costs?

The questions define an experimental chain:

$$\text{available} \rightarrow \text{discovered} \rightarrow \text{ranked} \rightarrow \text{materialized} \rightarrow \text{integrated} \rightarrow \text{decision} \rightarrow \text{behavior}. \quad (4)$$

An end-task score observes only the last part of this chain. Diagnosing a memory system requires measurements and interventions at the intermediate boundaries.

1.4 Experimentally Forced Distinctions

Table 1: Distinctions that prevent a positive result at one boundary from being attributed to another.

Distinction	Why it matters
Logical availability $\neq$ active computation	A large backing store can coexist with a small, bounded attention operation.
Encoding granularity $\neq$ routing granularity	Sources may require model-safe encoding blocks while smaller gists provide the useful search resolution.
Position $\neq$ contextual representation	Correct logical coordinates repair reset position; they do not restore cross-boundary contextualization lost during independent encoding.
Attention geometry $\neq$ semantic retrieval geometry	Native Q/K is optimized for token interaction after selection, not necessarily for ranking evidence before selection.
Discovery $\neq$ integration	An oracle can diagnose whether the consumer works when the learned router misses.
Integration $\neq$ decision formation	Memory may alter hidden states or average loss without moving answer-critical logits in the right direction.
Decision formation $\neq$ answer realization	Correct local polarity can coexist with a truncated, malformed, or otherwise incomplete generated answer.
Teacher-forced likelihood $\neq$ free-running behavior	Token loss and generated usefulness must be reported separately.
Lexical match $\neq$ behavioral utility	Exact match may miss useful paraphrases, while fluent overlap can conceal a wrong relation.
One-shot relevance $\neq$ relational closure	An initial relevant entity may identify the next necessary source only after it has been retrieved.
Selected relevance $\neq$ required detail	A selected chunk can be relevant even when only a small subset of its native K/V is causally necessary.

1.5 How to Read the Comparisons

Each method account follows the same sequence. It begins with the pressure that motivated the method, gives an intuition and formal mechanism, traces the resulting data flow, and then discusses evidence and failure modes. The final paragraph asks how to compare that method with PRA without changing several variables at once. These proposed comparisons are experimental designs, not reports of completed experiments.

Claims also have different statuses. Statements about prior systems are supported by the cited primary work. *Implemented* denotes a tested software mechanism. *Measured* denotes an experiment with a stated model, dataset, and intervention. *Proposed* denotes an architecture or experiment not yet validated. In particular, bounded one-shot native-KV PRA is measured; iterative associative closure and selective within-chunk materialization are proposed. Keeping these labels visible prevents software capability, oracle performance, and future architecture from being read as the same claim.

2 Progressive Retrieval Attention

2.1 PRA as a Worked Decomposition

Progressive Retrieval Attention (PRAAttention, PRA) begins with a separation that ordinary prompt construction leaves implicit: information may be *logically available* without participating in every model operation. A prompt can advertise a document, prior prompt prefix, code object, or record through a stable handle. The runtime can retain a compact search view for many such objects while admitting full-detail memory for only a selected working set. The model therefore sees two context sizes:

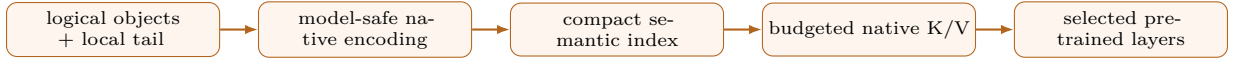
$$N_{\text{logical}} = \left| \bigcup_{u \in \mathcal{U}} z_u \right|, \quad N_{\text{active}}(t, \ell) = T_{\text{local}} + \sum_{c \in A_{t, \ell}} |c|, \quad (5)$$

where z_u is the source bound to URI u , $A_{t, \ell}$ is the set of chunks active at step t and layer ℓ , and T_{local} is the direct sequence length. Bounded execution requires a configured upper bound on N_{active} , not a promise that the logical store is small.

This is an access contract with five operations:

1. **name**: a handle identifies an object or a request-local prompt prefix;
2. **resolve**: a policy-aware resolver binds identity and version to content;
3. **index**: compact gists make chunks searchable without treating the gist as the evidence itself;
4. **materialize**: selected chunks expose full-detail model-native K/V; and
5. **consume**: selected native K/V enters chosen pretrained attention layers.

PRA’s novelty claim is this decomposition and its bounded composition, not the invention of top- k , K/V caching, cross-attention, or retrieval. The position, standalone, and pretrained studies evaluate different pieces of the contract [Simão, 2026a,d,c,b]. They do not establish a universal advantage over long context or RAG.



2.2 Identity, Resolution, and Bounded Logical Memory

Let the context graph be $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. A node $v = (u, z, m, R, \nu)$ contains URI u , content z , metadata m , an explicit child-reference table R , and version ν . An edge exists because a reference table names a child, not because two URI strings share a prefix. A resolver remains outside the neural layer:

$$\rho(u, p, t) \longrightarrow (z, m, R, \nu), \quad (6)$$

where p is an authorization domain and t is logical time. The separation permits files, database rows, prompt history, and document spans to share an interface without pretending that they share a physical store.

A cache key for reusable neural payload must bind at least URI, content version, model, consumer layer, encoding policy, positional provenance, and authorization domain. Otherwise identical names can alias different content or a K/V tensor can be consumed under a model or coordinate system that did not create it. Request-private caches avoid cross-request leakage but do not demonstrate shared reuse. Persistent reuse therefore adds invalidation, tenancy, and provenance obligations beyond the attention mechanism.

Long prompts use the same contract without requiring an explicit document URI. The recent tail remains in direct causal attention; an older prefix becomes a request-local implicit `__head__` object with its own chunk, routing, and budget parameters. This makes old prompt information recoverable without keeping all of it active. It does not extend the backbone’s learned positional range, and generated-token rollover is a separate streaming policy.

Recursive graph construction is bounded by shared object, depth, branch, source-token, and materialization budgets. Child-first construction allows a parent representation to use completed child memory where configured, while cycle states prevent unbounded traversal. These mechanics establish safe graph handling. A learned policy that follows evidence from one retrieved object to another has not yet been established.

2.3 Encoding Granularity, Routing Granularity, and Native Payload

One source partition cannot optimize every stage. An *encoding block* must fit the backbone’s model-safe context and preserve enough neighboring text for useful hidden states. A smaller *routing chunk* supplies search resolution. Several routing chunks may point into one contiguous encoded block, and their selected spans recover the corresponding native keys and values. Thus independent 32-token encoding and 32-token routing is not equivalent to contiguous 512-token encoding followed by 32-token routing.

For layer ℓ , source block hidden states $H_c^{(\ell)}$ are projected through the pretrained attention module:

$$K_c^{(\ell)} = \text{RoPE}_{p_c} \left(H_c^{(\ell)} W_K^{(\ell)} \right), \quad V_c^{(\ell)} = H_c^{(\ell)} W_V^{(\ell)}. \quad (7)$$

These tensors retain the native K/V head layout. The runtime does not insert a separate memory-value projection in the default path. After selection, it concatenates admitted native payload with local native K/V and invokes the model’s ordinary eager attention function with a mask that preserves local causality and permits access to memory:

$$\tilde{K}^{(\ell)} = [K_A^{(\ell)}; K_{\text{local}}^{(\ell)}], \quad \tilde{V}^{(\ell)} = [V_A^{(\ell)}; V_{\text{local}}^{(\ell)}], \quad (8)$$

$$O^{(\ell)} = \text{NativeAttn} \left(Q_{\text{local}}^{(\ell)}, \tilde{K}^{(\ell)}, \tilde{V}^{(\ell)}, \tilde{M} \right). \quad (9)$$

The disabled branch delegates to the unmodified attention module. Exact-bypass tests are therefore a compatibility gate: no selected memory must yield the original logits, not an approximation to them. For fixed selected identities, the tested K, V, and native attention output are bit-exact in the positional study [Simão, 2026c].

This native path differs from the historical standalone cross-attention variant. That variant encoded reference K/V through a distinct memory branch and output projection; it is useful as an adaptation control but breaks native-payload equivalence. The survey treats it as a secondary design point rather than PRA’s default transport.

2.4 Gists and Search Geometry

Routing should inspect a representation much smaller than full payload. For chunk c , a simple gist partitions hidden states into G contiguous subchunks and mean-pools each:

$$g_{c,j} = \frac{1}{|S_{c,j}|} \sum_{i \in S_{c,j}} h_i, \quad j \in \{1, \dots, G\}. \quad (10)$$

A query q scores each chunk by maximum, mean, or log-sum-exp aggregation over its gists. Reference-first routing can rank URI-level gists before chunk gists. Fixed- k bounds the absolute payload; fixed-fraction selection holds sparsity pressure approximately constant as the candidate count grows. Both should be reported because a constant k becomes a harder retrieval task at larger scales.

The important lesson from positional and pretrained experiments is that a native attention key is not automatically a good semantic index. Centered multi-gists increasingly reproduce native token-Q/K ranking, yet their semantic retrieval quality remains almost flat. A learned 32-dimensional asymmetric metric shows the reverse: strong semantic ranking with low native Q/K fidelity. The robust decomposition is therefore

$$\underbrace{r_\phi(q, g_c)}_{\text{semantic discovery}} \longrightarrow \underbrace{(K_c, V_c)}_{\text{native execution}}. \quad (11)$$

The router may use learned semantic geometry while the selected payload remains untouched native K/V. This separation also prevents a lossy routing summary from silently becoming the memory content.

2.5 Position Is Not Context

Independently encoded chunks introduce two different errors. *Positional reset* assigns each chunk positions beginning at zero. A source-relative logical offset repairs this error for learned absolute, sinusoidal, and RoPE coordinates. For RoPE, cached pre-rotation keys can also be rebound algebraically to a new offset, although deferred rotation has a measured runtime cost.

Contextual fragmentation remains after coordinates are repaired. A token encoded without its left source context has a different hidden state from the same token inside the contiguous document. Overlap or larger grouped encoding blocks reduce this mismatch at added encoding and storage cost. The resulting rule is simple: preserve source-relative coordinates, but do not describe coordinate correctness as recovered contextualization.

The measured geometry results make the distinction concrete. Source-relative offsets remove layer-zero reset error in all paired tests and improve deeper representations. Increasing centered-RoPE gists from one to eight raises native Q/K fidelity from .794 to .963, while semantic sparse AUC remains approximately .651. A learned router reaches .971 semantic AUC at only .178 Q/K fidelity. Position, contextual representation, semantic search, and native attention are four related but distinct objects.

2.6 Depth, Integration, Decision, and Behavior

Correct routing supplies evidence; it does not guarantee that a pretrained decoder knows how to use that evidence. PRA can be enabled at selected layers $\mathcal{L}_{\text{PRA}}$. Pretrained experiments find a non-monotone depth effect: a late layer

band can improve memory use, while injection into every layer can be disruptive. This turns layer placement into an integration variable rather than an implementation detail.

An oracle intervention replaces only selected identities while preserving transport, budget, and consumer. If oracle memory helps while routed memory does not, discovery is the bottleneck. If oracle memory also fails, representation or integration is implicated. QASPER provides a measured instance of this ladder: answer-polarity success progresses from 25.0% under frozen routed PRA to 62.5% under frozen oracle memory and $72.5 \pm 5.6\%$ with a small residual adapter. Blind behavioral judges also favor the residual-adapted QASPER outputs. The result shows that one-shot memory can help and that consumer calibration still matters.

The same experiment also shows why likelihood is insufficient. A model can improve teacher-forced loss without completing an answer, and a token-level polarity improvement can coexist with truncation. Evaluation therefore needs answer-critical logit margins, completion rate, lexical metrics, task semantics, and blinded behavioral comparison. On HotpotQA, outputs often approach a relevant entity or relation but miss another required hop or endpoint. This is evidence of a one-shot relational boundary, not proof that more decoder adaptation alone will solve multi-hop discovery.

2.7 Evidence Ledger

Table 2 summarizes the current empirical boundary. Results come from different model scales and protocols and should not be pooled as one benchmark score. Their role in this survey is to identify which links in the access chain have been tested.

Table 2: PRA evidence by computational boundary. “Measured” does not imply production validation or universal superiority.

Boundary	Measured result	Interpretation and limit
Ordinary path	Disabled adapters preserve the host path exactly across standalone and pretrained-family tests	Compatibility gate; it says nothing about memory utility
Position	Source-relative offsets repair reset fragmentation; native pre/post/deferred RoPE paths agree under matched coordinates	Coordinate correctness is established; contextual fragmentation remains
Logical versus active	At 64–256 addressable units, fixed $k = 8$ holds physical selected K/V near 11–13 tokens and reduces active fraction to 4.4–4.9% at 256	Bounded native transport on controlled probes, not unrestricted long-document QA
Routing scale	At about 12.5% selection, target coverage remains .756–.772 on HotpotQA-derived and .794–.803 on QASPER-derived probes through 256 units	Stable fixed-fraction quality; substantial router headroom remains
Search geometry	Centered $G = 1 \rightarrow 8$ improves Q/K fidelity .794 $\rightarrow$.963 with flat semantic AUC; learned 32-D routing reaches .971 AUC	Semantic search should be separated from native execution
Pretrained retrofit	Qwen, Llama-family, and Gemma adapters preserve disabled semantics; selected native K/V uses family-native head layouts	Cross-family mechanical portability at roughly one-billion-parameter scale, not production coverage
Depth and use	Late-band memory is useful; indiscriminate all-layer activation is harmful in the measured setting	Integration is depth-sensitive
One-shot QASPER	Routed frozen 25.0%, oracle frozen 62.5%, residual adapter $72.5 \pm 5.6\%$ polarity; behavioral judges favor adaptation	Positive one-shot regime on a small fixed cohort
One-shot HotpotQA	Relation-near outputs frequently miss another required evidence item or endpoint	Motivates iterative closure; does not validate it
Serving	Packed tensor routing uses vectorized scoring and topk ; 256-way warm selection is 5.5–11.6 ms in the standalone prototype	Better than exhaustive scalar routing, but no optimized serving-kernel claim

2.8 Systems Accounting

Let G be the number of routing gists, M selected native K/V tokens, L_p PRA-enabled layers, and w bytes per scalar. Ignoring head constants, exact dense gist scoring costs $O(Gd)$ and active memory attention costs $O(L_p T_{\text{local}} M d)$. Resident selected payload costs approximately

$$B_{\text{active}} \simeq 2wL_p M d, \quad (12)$$

while the backing cache may be larger and may reside on CPU or another storage tier. Reducing active attention does not automatically reduce total storage. Likewise, a short prompt is not an efficiency gain if every source is cold-encoded for every request.

A systems report must separate source resolution, tokenization, model-safe encoding, index construction, routing, host-to-device transfer, native attention, synchronization, and generation. It should report cold, warm, and authorized shared-cache regimes; logical tokens; available and selected gists; physical selected K/V; peak accelerator and host bytes; time to first token; inter-token latency; and avoided encoding work. Packed **torch.topk** is a prototype optimization, not evidence of an asymptotically indexed router or fused sparse attention kernel.

2.9 Causal Evaluation Doctrine

For loss L , define

$$\Delta_{\text{use}} = L_{\text{disabled}} - L_{\text{valid}}, \quad \Delta_{\text{content}} = L_{\text{shuffled}} - L_{\text{valid}}. \quad (13)$$

A positive Δ_{use} only shows that the active branch changes the model usefully. Content-causal retrieval additionally requires valid evidence to outperform shuffled, irrelevant, stale, and matched-length controls. Oracle identity measures the upper bound of the same consumer; direct text measures whether the backbone can solve the item when evidence is local. All-reference and gist-only conditions isolate routing and detail bottlenecks.

These controls should be crossed with routing budget, candidate count, layer band, and materialization mode. Report reference recall, all-required-evidence recall, physical K/V fraction, answer-critical logit margins, task output, and free-running behavior. Attention weights and lexical overlap are diagnostics, not causal proof.

2.10 Architecture Ladder: Established, Next, and Later

The PRA research program now has three distinct levels.

Established in the current evidence. Bounded logical memory, model-safe source encoding, separate routing chunks, semantic gist routing, full-detail native-KV payload, pretrained retrofit with exact disabled paths, and depth-sensitive integration are implemented and measured. The evidence is strongest for mechanical fidelity, controlled scaling, routing diagnostics, and a small one-shot QASPER regime.

Next: associative closure over gists. One-shot routing computes $C_1 = R(q_0, \mathcal{I})$. A proposed bounded iterative policy lets retrieved gist content update the query and expose another relation:

$$C_h = R(q_{h-1}, \mathcal{I} \setminus V_{h-1}), \quad (14)$$

$$q_h = U(q_{h-1}, G(C_h)), \quad h = 1, \dots, H, \quad (15)$$

$$A = \text{BudgetSelect} \left(\bigcup_{h=1}^H C_h, B \right). \quad (16)$$

Here V_{h-1} prevents cycles, H bounds hops, and B fixes the final unique native-KV budget. The retrieved objects form an evidence path rather than an unordered bag. HotpotQA is the natural primary test because the current failure is often a missing relation or endpoint; QASPER is a one-shot control. This is *associative closure*: a retrieved concept helps discover another source. It is not yet an empirical PRA result and should not be described as full neural reasoning.

Later: progressive and smart materialization. After relevant chunks are found, a second policy may decide which native detail to admit. Partial pages, progressive escalation, token-level bounds, and head/layer-specific budgets are possible mechanisms. Under the **redundancy hypothesis**, less detail yields equivalent quality because much selected K/V is unnecessary. Under the **overload or dilution hypothesis**, less detail improves quality because irrelevant detail competes with causal evidence, a concern consistent with positional sensitivity in long contexts [Liu et al., 2024b]. If active K/V falls while quality is unchanged, the first explanation is supported. If quality improves, the second becomes plausible but still requires content-removal controls. If quality falls, omitted detail was useful or the allocator was wrong. These are Paper 3 hypotheses, not consequences of the one-shot results.

The ladder preserves the central thesis without overextending it: PRA is an emerging decomposition of bounded memory computation. It has established a native one-shot working-set mechanism; it has not yet established relational closure, optimal materialization, or a production serving advantage. The next seven sections apply the access-contract vocabulary to representative systems. To keep the mathematics comparable, T denotes sequence length, d model width, and d_h the width of one attention head. The matrices Q , K , and V contain projected queries, keys, and values; M_{causal} masks positions that a causal decoder may not read. Method-specific symbols are introduced where they appear. Complexity expressions describe the dominant conceptual operation and omit constant head and implementation factors unless those factors are themselves the subject of the method.

3 Category I: Dense, Sparse, and Approximate Long-Context Attention

This family changes the token-interaction graph after information has already been materialized as one sequence. It is therefore the cleanest “all evidence local” control for PRA: selection is either absent or encoded in the attention mask, object identity is reduced to position, and no cross-request materialization is implied.

3.1 Dense attention and positional extension

Motivation and mechanism. Dense causal attention is the least structured but most general access rule: every position can read every earlier position. Rotary position embeddings encode relative offsets through query/key rotations [Su et al., 2021]; position interpolation and long-context continued pretraining can extend the usable position range without changing this graph [Chen et al., 2023b]. For T tokens,

$$A = \text{softmax}((QK^\top + M_{\text{causal}})/\sqrt{d_h})V, \quad \text{time } O(T^2d), \text{ attention storage } O(T^2). \quad (17)$$

Positional extrapolation and attention efficiency are different problems. A model may accept a long index while still allocating quadratic work, or execute efficiently while failing to use distant positions.

Listing 1: Dense causal attention; PyTorch uses optimized kernels when possible

```
def dense_causal(q, k, v): # [B,H,T,D]
    return F.scaled_dot_product_attention(q, k, v, is_causal=True)
```



Evidence, failure modes, and lineage. Standard language modeling and long-context adaptations use perplexity, long-document QA, retrieval probes, and throughput/memory sweeps. Dense attention is a strong baseline when sources are short or densely relevant, and modern fused kernels make it unexpectedly competitive at moderate lengths. Its failures include quadratic cost, distraction, positional degradation, and lack of stable object identity or reuse beyond exact prefixes. Needle tests diagnose reachability but do not establish synthesis or citation behavior.

Relation and comparison to PRA. Dense context performs eager materialization; PRA advertises a potentially larger logical context and materializes selected objects. Compare them on the same source corpus under (i) equal visible-plus-retrieved tokens, (ii) equal FLOPs, (iii) equal peak KV bytes, and (iv) equal synchronized latency. Sweep evidence density and object reuse. Dense attention should be expected to win when most tokens matter once; PRA’s hypothesis is strongest when few named objects serve many queries. The all-content-local condition is also PRA’s transport upper control.

3.2 Longformer

Motivation and mechanism. Longformer observes that document dependencies are mostly local but a few task positions need global connectivity. A token attends within a window and, optionally, to a set $\mathcal{G}$ of global positions [Beltagy et al., 2020]:

$$\mathcal{N}(i) = \{j : |i - j| \leq w/2\} \cup \mathcal{G}, \quad O(Tw + T|\mathcal{G}|). \quad (18)$$

Local layers propagate information gradually; global tokens create short paths across the document and can be task-selected (for example, question tokens in QA). Unlike truncation, the model retains an explicit sparse graph over the entire input.

Listing 2: Educational Longformer mask; production kernels avoid materializing T^2

```
def longformer_mask(T, window, global_pos, device):
    i = torch.arange(T, device=device)
    local = (i[:, None] - i[None, :]).abs() <= window // 2
    g = torch.zeros(T, dtype=torch.bool, device=device)
    g[global_pos] = True
    return local | g[:, None] | g[None, :]
```



Evidence and paper trail. The original paper studies character language modeling, long-document classification, and QA; its encoder-decoder continuation LED applies the pattern to long-input generation. Comparisons include RoBERTa-style dense baselines and task-specific long-document systems, using bits-per-character, accuracy/F1, and generation metrics. Longformer became a template for domain adaptations and local/global attention. Its main failure is structural: useful long-range routes must be known through the global mask or emerge through multiple local hops. Fixed windows can split tables, citations, or cross-section evidence, and global positions can become a bottleneck.

PRA relation and matched experiment. Longformer makes relevance a sparse edge decision over already materialized tokens; PRA makes it an object-admission decision before latent transport. Give both the same documents and vary evidence sparsity, distance, and whether section boundaries are known. Match the number of attended key positions: Longformer local/global edges versus PRA top- k object tokens. Add shuffled global markers and shuffled URI content. This separates sensitivity to graph placement from sensitivity to object semantics. A hybrid can use Longformer locally inside a large resolved object and PRA between objects.

3.3 BigBird

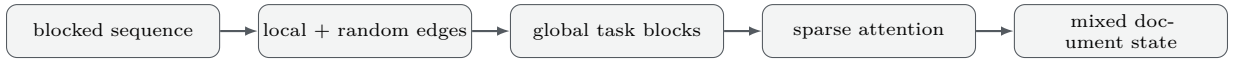
Motivation and mechanism. BigBird combines window, random, and global edges so that each token has bounded degree while the graph mixes rapidly [Zaheer et al., 2020]:

$$\mathcal{N}(i) = \mathcal{N}_{\text{window}}(i) \cup \mathcal{N}_{\text{random}}(i) \cup \mathcal{G}. \quad (19)$$

The random component supplies shortcuts without content-dependent routing; global tokens connect task anchors to all positions. Under the paper’s graph assumptions this pattern retains strong expressivity results while reducing sequence complexity to linear in T for fixed block sizes.

Listing 3: Block-level BigBird adjacency sketch

```
def bigbird_blocks(n_blocks, radius, random_k, global_blocks=(0,)):
    edges = []
    for i in range(n_blocks):
        local = range(max(0, i-radius), min(n_blocks, i+radius+1))
        random = torch.randperm(n_blocks)[:random_k].tolist()
        edges.append(sorted(set(local) | set(random) | set(global_blocks)))
    return edges
```



Evidence, ablations, and limitations. BigBird evaluates natural-language QA and summarization as well as genomic sequences, comparing dense BERT/Roberta-style models and other efficient Transformers with task accuracy/F1, ROUGE, and sequence-scale behavior. The meaningful ablation is the contribution of local, random, and global edges. Random connectivity is inexpensive on average but can miss a particular necessary route; global tokens require a placement rule, and sparse kernels do not automatically make an otherwise identical pretrained model work at new lengths.

PRA relation and experiment. BigBird constructs content-independent shortcuts inside one anonymous token graph. PRA constructs content-dependent edges to explicitly named external objects. Compare a matched sparse-edge budget in which random/global block edges and top- k URI objects expose the same number of keys. Sweep distractor blocks, wrong handles, evidence distance, and repeated queries. Report cold and warm materialization for PRA; otherwise BigBird is unfairly charged only for attention while PRA hides encoding. An instructive hybrid replaces random inter-document edges with URI-restricted routing and retains BigBird within each selected document.

3.4 Performer

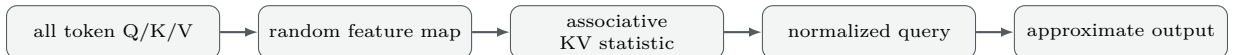
Motivation and mechanism. Performer approximates softmax attention with positive random features (FAVOR+), exchanging a $T \times T$ matrix for associative summaries [Choromanski et al., 2021]. If $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$, then

$$\text{Attn}(Q, K, V) \approx D^{-1} \phi(Q) (\phi(K)^\top V), \quad D = \text{diag}(\phi(Q) (\phi(K)^\top \mathbf{1})). \quad (20)$$

For fixed feature count, time and memory are linear in sequence length. Causal execution uses prefix sums of $\phi(k_i)v_i^\top$ and $\phi(k_i)$.

Listing 4: Causal kernel-attention recurrence; phi must be a positive feature map

```
def causal_linear_attention(qf, kf, v, eps=1e-6):
    kv = torch.einsum('...tm,...td->...tmd', kf, v).cumsum(dim=-3)
    kz = kf.cumsum(dim=-2)
    num = torch.einsum('...tm,...tmd->...td', qf, kv)
    den = torch.einsum('...tm,...tm->...t', qf, kz).unsqueeze(-1)
    return num / den.clamp_min(eps)
```



Evidence, failure modes, and lineage. Performer reports image, text, and protein experiments and measures both downstream quality and approximation/efficiency. Its paper trail lies in kernelized linear attention and random-feature estimation. Feature variance, finite-precision instability, causal implementation details, and inability to recover a specific token after destructive aggregation are central failure modes. Linear cost does not mean selective relevance: every token contributes to the statistic.

PRA relation and comparison. Performer approximates computation after eager materialization; PRA changes which objects enter computation. The two are complementary: use linear attention for the local stream or within

a resolved memory. A fair experiment crosses exact versus FAVOR+ local attention with all-local versus PRA access, reports approximation error and exact-value recall, and sweeps random-feature count under fixed FLOPs. PRA corruption tests remain necessary because a low approximation error says nothing about whether the correct reference was selected.

4 Category II: Recurrence and Compression

Recurrence changes the temporal boundary: earlier segments leave behind state rather than remaining as raw local tokens. The central distinction from PRA is *write-time retention versus read-time dereferencing*. Recurrent systems decide what history survives as it passes; PRA assumes an addressable backing object can be fetched later.

4.1 Transformer-XL

Motivation and mechanism. Transformer-XL addresses fixed context and segment fragmentation by carrying the previous segment’s hidden states forward as stop-gradient memory and using relative positional attention [Dai et al., 2019]:

$$\tilde{H}_s^{(\ell-1)} = [\text{SG}(H_{s-1}^{(\ell-1)}); H_s^{(\ell-1)}], \quad Q = H_s W_Q, \quad K, V = \tilde{H}_s W_{K,V}. \quad (21)$$

Only current-segment states receive gradients through the recurrent boundary. The memory extends effective context without backpropagating across the entire history; relative positions prevent a cached state from being confused with a current absolute position.

Listing 5: Segment recurrence; detach is the defining optimization boundary

```
def transformer_xl_segment(layer, x, old_memory, memory_len):
    kv_source = torch.cat([old_memory.detach(), x], dim=1)
    y = layer(query=x, key=kv_source, value=kv_source, relative_positions=True)
    new_memory = torch.cat([old_memory, x], dim=1)[: , -memory_len:].detach()
    return y, new_memory
```



Evidence and limitations. The paper evaluates WikiText-103, One Billion Word, Penn Treebank, enwiki8, and text8 using perplexity or bits-per-character, alongside recurrent and Transformer language-model baselines. It reports then-leading benchmark results and substantially faster evaluation than recomputing overlapping windows. Ablations isolate recurrence and relative positions. Memory is chronological, anonymous, fixed-length, and not revisable: evicted information is gone, irrelevant history consumes capacity, and a detached boundary limits credit assignment.

PRA relation and direct test. Transformer-XL answers “what recent state should remain?”; PRA answers “which named remote object should be loaded now?” Compare them on streams with either recency-correlated evidence or remote named evidence. Match resident KV bytes and total encoding FLOPs, sweep distance beyond the recurrent window, and include repeated access to the same old object. A hybrid keeps Transformer-XL memory for recent discourse while letting a URI handle reload evicted or non-contiguous source material.

4.2 Compressive Transformer

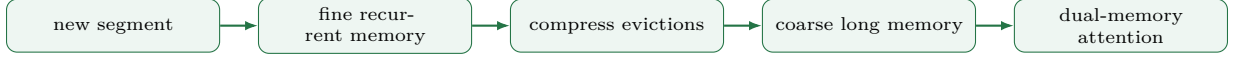
Motivation and mechanism. Compressive Transformer extends Transformer-XL with a second, lower-resolution memory. States evicted from fine memory M^m are transformed by a compressor C before entering compressed memory M^c [Rae et al., 2020]:

$$M_s^c = \text{tail}_{n_c}([M_{s-1}^c; C(E_{s-1})]), \quad (22)$$

where E_{s-1} denotes the fine states being evicted. Attention reads both stores. Pooling, convolution, and usage-based compression trade temporal resolution for longer reach; auxiliary reconstruction losses encourage retained predictive information.

Listing 6: Two-tier chronological memory update

```
def update_memory(fine, compressed, new_states, fine_cap, comp_cap, compress):
    merged = torch.cat([fine, new_states], dim=1)
    evicted, fine = merged[:, :-fine_cap], merged[:, -fine_cap:]
    compressed = torch.cat([compressed, compress(evicted)], dim=1)
    return fine.detach(), compressed[:, -comp_cap:].detach()
```



Evidence and failure modes. The original work studies long-range language modeling, compares compression functions and auxiliary objectives, and uses perplexity plus attention and memory diagnostics. The method’s honest trade-off is rate versus fidelity. Pooling can erase identifiers, superposition can merge distractors, learned compression can optimize average prediction while losing rare facts, and chronological stores cannot directly invalidate a stale semantic object.

PRA relation and experiment. PRA ordinarily reloads an original or cached object; Compressive Transformer obligatorily summarizes passing history. A useful hybrid attaches several materializations to one URI: metadata, summary, compressed latent, and full KV, with uncertainty-driven escalation. Compare fine-only recurrence, dual-memory compression, summary-only PRA, and full-object PRA under equal resident bytes. Sweep compression ratio, rare exact facts, semantic synthesis, and stale-object replacement. Report whether escalation recovers failures and charge full-object encoding in cold-cache trials.

4.3 Recurrent Memory Transformer (RMT)

Motivation and mechanism. RMT inserts a small set of learned memory tokens into every segment. Their output states become the memory-token inputs to the next segment, creating an end-to-end learned bottleneck [Bulatov et al., 2022]. If m_s are incoming memory tokens and x_s current tokens,

$$[\hat{m}_s, H_s, m_{s+1}] = \text{Transformer}([m_s, x_s, m_s^{\text{write}}]). \quad (23)$$

The exact placement varies by implementation, but the conceptual operation is stable: read old slots, process the segment, and write a fixed number of new slots.

Listing 7: RMT segment interface

```
def rmt_step(model, tokens, memory, write_tokens):
    sequence = torch.cat([memory, tokens, write_tokens], dim=1)
    hidden = model(sequence)
    next_memory = hidden[:, -write_tokens.size(1):]
    token_hidden = hidden[:, memory.size(1):-write_tokens.size(1)]
    return token_hidden, next_memory
```



Evidence, lineage, and failures. RMT is evaluated on language modeling and algorithmic/long-sequence tasks; later work demonstrates extrapolation on very long synthetic problems. Baselines include finite-window Transformers and recurrent-memory variants, with loss, task accuracy, and length extrapolation as metrics. The learned bottleneck is simple and differentiable, but it must anticipate future needs at write time. Rare details can be overwritten, slot semantics lack stable provenance, and backpropagation depth or memory curricula strongly affect training.

PRA relation and experiment. RMT learns *what to remember while reading*; PRA learns *what to fetch while answering*. Compare both on matched memory bytes under two datasets: one where future questions are predictable during ingestion and one where they are not. Substitute or reset RMT slots and shuffle PRA references to measure content use. A hybrid stores an RMT summary in the URI metadata tier but preserves the source object for read-time expansion when the summary is insufficient.

4.4 Block-Recurrent Transformer

Motivation and mechanism. Token-wise recurrence is sequential, whereas ordinary self-attention is parallel but bounded by its active window. Block-Recurrent Transformer places the recurrent boundary at a block of tokens. A

set of state vectors S_{b-1} and the current token block X_b exchange information through self- and cross-attention, then LSTM-style gates form the state passed to the next block [Hutchins et al., 2022]:

$$\tilde{X}_b = \text{TokenBlock}(X_b, S_{b-1}), \quad (24)$$

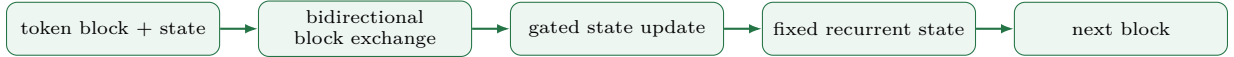
$$\tilde{S}_b = \text{StateBlock}(S_{b-1}, X_b), \quad (25)$$

$$S_b = g_b \odot S_{b-1} + (1 - g_b) \odot \tilde{S}_b. \quad (26)$$

The exact cell contains several gates, but the computational principle is visible here: parallel token processing occurs inside a block, while a fixed set of latent states carries information between blocks. For fixed block and state sizes, total work grows linearly with the number of processed tokens.

Listing 8: Simplified block-recurrent update

```
def block_recurrent_step(token_block, state, cell):
    token_out = cell.tokens_attend(token_block, state)
    state_candidate = cell.state_attends(state, token_block)
    gate = torch.sigmoid(cell.gate(torch.cat([state, state_candidate], -1)))
    next_state = gate * state + (1 - gate) * state_candidate
    return token_out, next_state
```



Evidence and limitations. The original study evaluates PG19 books, arXiv papers, and GitHub source code against long-range Transformer-XL variants, reporting language-model loss and execution behavior. The design uses accelerator-friendly block parallelism, but the state remains an anonymous write-time bottleneck. A gate can preserve a feature without preserving its source, version, or exact wording, and later queries cannot reload discarded detail.

PRA relation and experiment. Block recurrence and PRA solve different halves of the working-set problem. The recurrent state keeps a compact summary of the path already read; PRA can dereference a named object that was never read or whose details no longer fit the state. Compare recurrent state, PRA, and a hybrid under matched resident bytes on tasks that independently vary temporal continuity and source addressability. State resets test dependence on recurrent content; shuffled URI objects test dependence on PRA content.

4.5 TransformerFAM

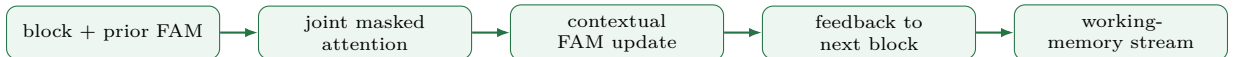
Motivation and mechanism. A sliding-window transformer eventually loses access to every old token, and a detached segment cache limits how gradients teach long-range retention. TransformerFAM introduces a short Feedback Attention Memory (FAM) whose current activations attend with each token block and whose updated activations feed the next block [Hwang et al., 2024]. If F_{b-1} is the previous feedback state, one abstract update is

$$[H_b, F_b] = \text{MaskedAttn}([X_b, F_{b-1}], [X_b, M_b, F_{b-1}]), \quad (27)$$

where M_b denotes the ordinary block-sliding memory. The feedback queries are contextual: they change after every block instead of using one static summary query. FAM adds no new projection weights when it reuses the transformer’s attention parameters, although it changes training, masks, positions, and recurrent execution.

Listing 9: Feedback attention memory across blocks

```
def fam_step(block, sliding_memory, fam, attention, mask):
    queries = torch.cat([block, fam], dim=1)
    keys_values = torch.cat([block, sliding_memory, fam], dim=1)
    outputs = attention(queries, keys_values, keys_values, attn_mask=mask)
    return outputs[:, :block.size(1)], outputs[:, block.size(1):]
```



Evidence and limitations. The paper studies models at 1B, 8B, and 24B parameters on language modeling, passkey retrieval, NarrativeQA, and related long-context tasks. Its ablations show that static summary queries and some simpler feedback variants do not retain the passkey as well as contextual FAM. Improvements over block sliding attention can be modest on several natural tasks, and the latent slots still lack stable object identity, selective invalidation, or exact reconstruction after compression.

PRA relation and experiment. FAM is dynamic working memory; PRA is addressed backing memory. A useful hybrid lets FAM carry the current reasoning state while PRA supplies source detail. Compare a static summary token, contextual FAM, content-gist PRA, and the hybrid. Use delayed exact recall, multi-hop synthesis, stale-source replacement, and unseen question types. The key question is whether a small feedback state reduces PRA faults without hiding content failures inside an uninterpretable latent bottleneck.

4.6 Infini-attention

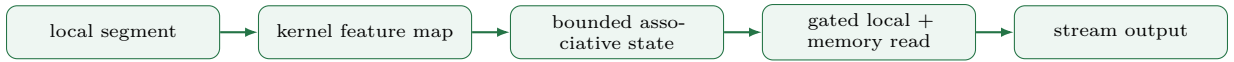
Motivation and mechanism. Infini-attention combines masked local attention with a bounded associative memory inside each block [Munkhdalai et al., 2024]. With positive feature map ϕ , a segment updates

$$M_s = M_{s-1} + \phi(K_s)^\top V_s, \quad z_s = z_{s-1} + \sum_i \phi(k_{s,i}), \quad (28)$$

and reads $\phi(Q_s)M_{s-1}/(\phi(Q_s)z_{s-1})$. A learned gate mixes this compressive read with dot-product attention over the current segment. Memory size is independent of total processed length, which makes streaming execution possible.

Listing 10: Associative-memory read and write

```
def infiniqu(qf, kf, v, M, z, eps=1e-6):
    read = torch.einsum('btd,bdh->bth', qf, M)
    denom = torch.einsum('btd,bd->bt', qf, z).unsqueeze(-1)
    read = read / denom.clamp_min(eps)
    M = M + torch.einsum('btd,bth->bth', kf, v)
    z = z + kf.sum(dim=1)
    return read, M.detach(), z.detach()
```



Evidence and limitations. The paper evaluates long-context language modeling, one-million-token passkey retrieval, and 500K-token book summarization with 1B- and 8B-scale models. Metrics include perplexity, retrieval accuracy, summarization quality, and memory scaling. Bounded state is the strength and the limitation: many keys superpose, collisions and normalization can obscure exact facts, and there is no explicit object version, authorization boundary, or selective invalidation.

PRA relation and direct comparison. Infini-attention offers constant-size lossy history; PRA offers selective identity with potentially unbounded backing storage and bounded resident working set. Match resident bytes and local window, then sweep number of objects, repeated keys, exact-value queries, semantic summaries, and stale replacements. PRA must include resolution/encoding latency. A hybrid maintains cheap Infini state for the stream while a collision, low-confidence query, or explicit handle faults in a named PRA object.

4.7 Titans and Test-Time Neural Memory

Motivation and mechanism. Fixed-state recurrent models update activations, but their update rule is usually shallow. Titans instead treats a neural network $\mathcal{M}_{\theta_t}$ as long-term memory and updates its parameters while processing the test sequence [Behrouz et al., 2025]. For a key-value association (k_t, v_t) , an associative loss measures how poorly the current memory predicts the value,

$$\ell_t(\theta_t) = \|\mathcal{M}_{\theta_t}(k_t) - v_t\|_2^2, \quad s_t = \|\nabla_{\theta_t} \ell_t\|, \quad (29)$$

and the gradient magnitude s_t acts as a surprise signal. Momentum and forgetting terms control how the parameter update incorporates surprising events. Titans combines this neural long-term memory with precise short-window attention and data-independent persistent memory. Its Memory-as-Context, Memory-as-Gate, and Memory-as-Layer variants differ in where the long-term read enters the sequence model.

Listing 11: Conceptual surprise-weighted test-time memory update

```
def neural_memory_step(memory, key, value, optimizer_state):
    prediction = memory(key)
    loss = F.mse_loss(prediction, value)
    gradients = torch.autograd.grad(loss, memory.parameters())
    surprise = global_norm(gradients)
    optimizer_state = momentum_and_forgetting(optimizer_state, gradients, surprise)
    apply_test_time_update(memory, optimizer_state)
    return prediction, surprise
```



Evidence and limitations. Titans reports language modeling, common-sense reasoning, genomics, time-series, and long-context retrieval experiments, including context lengths beyond two million tokens. Ablations examine memory depth, momentum, forgetting, convolution, persistent memory, and the three integration variants. These results motivate test-time neural memory, but online parameter updates introduce sequential optimization, state-reset,

reproducibility, privacy, and contamination questions. The memory is content addressed rather than an authoritative source store; forgetting or correcting one object is not a simple URI invalidation.

PRA relation and experiment. Titans learns a compressed latent history online; PRA resolves versioned external objects and transports selected evidence. Compare them on four operations: repeated semantic patterns, exact quotations, correction of stale facts, and access to an object never seen in the current stream. Cross both methods with state reset, corrupted evidence, and repeated-query conditions. A hybrid could use Titans for experience-dependent routing priors while PRA retains inspectable source identity. Such a hybrid must report whether test-time updates improve object selection or merely adapt the decoder independently of supplied content.

4.8 Selective State-Space Models

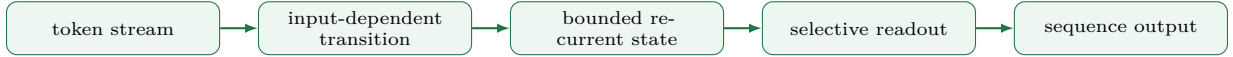
Motivation and mechanism. State-space sequence models replace pairwise token attention with a recurrent state whose update can be computed efficiently across a sequence. Mamba makes the state transition input dependent so the model can selectively propagate or forget information [Gu and Dao, 2024]. A schematic selective recurrence is

$$h_t = \overline{A}(x_t)h_{t-1} + \overline{B}(x_t)x_t, \quad y_t = C(x_t)h_t + Dx_t. \quad (30)$$

The state size is bounded with sequence length, and the hardware-aware scan supports parallel training and recurrent inference.

Listing 12: Conceptual input-selective state update

```
def selective_ssm_step(x_t, state, parameterizer):
    A_t, B_t, C_t = parameterizer(x_t)
    state = A_t * state + B_t * x_t
    return C_t * state, state
```



Evidence and limitations. Mamba reports competitive language, audio, and genomics results with linear sequence scaling. Its active state is compact and naturally supports streaming. Exact old evidence, however, is superposed in anonymous recurrent state rather than retained as an independently inspectable object. Correction, source-level invalidation, and random access are therefore different operations from updating a URI cache.

PRA relation and experiment. Selective state-space memory and PRA occupy different points on the exactness–addressability frontier. Match active bytes and throughput on streams that mix recurring patterns, exact delayed values, source corrections, and random object queries. A hybrid can use bounded state for routine chronology and fault named evidence into native-KV memory when exact or auditable detail is required.

5 Category III: Differentiable and Episodic External Memory

These methods move state outside the ordinary recurrent or token representation while keeping access near neural computation. They introduce two questions that recur in PRA: who writes the memory, and whether an address denotes a stable object or only a similar vector.

5.1 Memory Networks

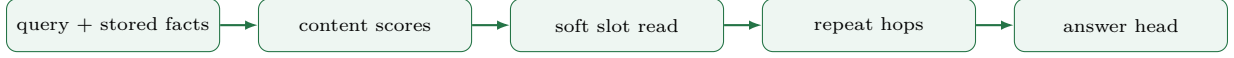
Motivation and mechanism. Memory Networks separate input encoding, memory storage, addressing, and response production [Weston et al., 2015]. Facts become slots m_i ; a query q produces a content read

$$p_i = \text{softmax}_i(q^\top m_i), \quad o = \sum_i p_i c_i, \quad q' = q + o. \quad (31)$$

Repeating the read implements multi-hop reasoning. End-to-end variants learn embeddings from question-answer supervision; earlier variants used stronger supporting-fact labels.

Listing 13: One differentiable memory hop

```
def memory_hop(query, address_slots, content_slots):
    weights = torch.softmax(query @ address_slots.transpose(-1, -2), dim=-1)
    read = weights @ content_slots
    return query + read, weights
```



Evidence, lineage, and failure modes. The paper trail runs from supervised Memory Networks through end-to-end memory networks and key-value memory networks. Evaluations include synthetic bAbI reasoning and question answering, using answer accuracy and supporting-fact retrieval. Multiple hops make compositional access inspectable, but soft attention across a large flat store scales poorly; embeddings can exploit lexical shortcuts; and overwrite, provenance, and slot lifecycle are under-specified for production knowledge.

PRA relation and experiment. Memory Networks establish the learned-read primitive that PRA reuses, but PRA delegates object writes and identity to a resolver/cache. Compare flat learned slots with URI candidate restriction followed by the same within-candidate attention. Hold stored facts and read hops fixed; sweep memory size, paraphrase, distractors, and wrong-object substitutions. Report supporting-fact recall and counterfactual answer change, not attention maps alone. The expected trade-off is open semantic discovery versus cheap exact restriction when a handle is available.

5.2 Differentiable Neural Computer (DNC)

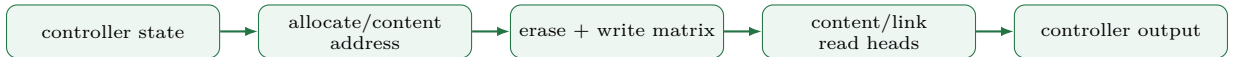
Motivation and mechanism. The DNC couples a neural controller to a read/write matrix with content lookup, temporal links, usage-based allocation, and differentiable erase/add operations [Graves et al., 2016]. A write updates row i as

$$M_t(i) = M_{t-1}(i) \odot (1 - w_t^w(i)e_t) + w_t^w(i)a_t, \quad (32)$$

while read weights mix content addressing with forward/backward traversal of a learned temporal linkage. It therefore supports both associative recall and ordered walks.

Listing 14: Core DNC erase/add write; allocation and link updates omitted

```
def dnc_write(memory, write_weight, erase, add):
    w = write_weight.unsqueeze(-1)
    return memory * (1 - w * erase.unsqueeze(1)) + w * add.unsqueeze(1)
```



Evidence and limitations. DNC experiments emphasize synthetic graph traversal, question answering, and algorithmic tasks, comparing recurrent controllers and earlier memory systems with exact task accuracy or sequence error. The machine can learn compact algorithm-like behavior, yet many interacting gates make training brittle. Soft access is expensive at large slot counts, learned addresses lack stable external meaning, and catastrophic write errors can corrupt later computation.

PRA relation and experiment. DNC learns both memory management and access; PRA externalizes durable writes, versions, and permissions, narrowing learning to selection and fusion. Compare DNC, read-only DNC, and PRA on a graph task whose nodes have stable IDs plus a variant with IDs hidden. Match active memory bytes and controller parameters. Test node substitution, temporal traversal, stale updates, and multi-tenant isolation. A hybrid can use a small DNC as transient scratch space while PRA supplies immutable, auditable backing objects.

5.3 Memformer

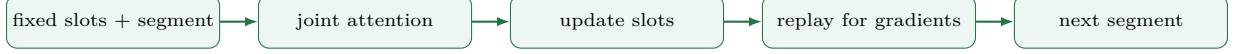
Motivation and mechanism. Memformer carries a fixed number of dynamic memory slots across segments and introduces memory replay backpropagation so training need not retain every old activation [Wu et al., 2020]. A segment attends to slots and local tokens, then a write operation updates the slots:

$$S_{t+1} = \text{Write}(S_t, H_t), \quad |S_t| = N_s \text{ constant.} \quad (33)$$

Replay reconstructs selected memory states during backward computation, trading extra compute for lower activation storage.

Listing 15: Simplified fixed-slot segment update

```
def memformer_step(block, tokens, slots):
    joint = torch.cat([slots, tokens], dim=1)
    hidden = block(joint)
    new_slots = hidden[:, :slots.size(1)]
    token_hidden = hidden[:, slots.size(1):]
    return token_hidden, new_slots
```



Evidence, ablations, and failures. The paper evaluates sequence modeling under long inputs, reporting task quality, training memory, and speed against Transformer and memory-augmented baselines. Slot count, recurrence, and replay are key ablations. Constant space is attractive, but slots are lossy summaries without stable semantic names; useful facts may be overwritten; replay adds complexity; and a slot cannot be independently revoked or refreshed when its source changes.

PRA relation and experiment. Memformer provides a learned working set; PRA adds a named backing store and explicit faults. Use identical segment encoders and match resident slots/KV bytes. Ask questions whose evidence is either predictable during ingestion or selected only after ingestion, and sweep time-to-query. Compare slot reset/substitution with PRA disabled/shuffled controls. A tiered design stores Memformer summaries as cheap URI metadata and faults in full layer-specific memory when summary confidence is low.

5.4 Memorizing Transformer

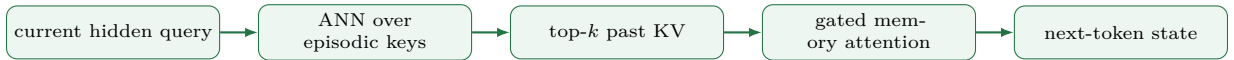
Motivation and mechanism. Memorizing Transformer records past key/value pairs in a large non-differentiable episodic database and retrieves nearest keys for current queries [Wu et al., 2022]. For query q ,

$$\mathcal{I} = \text{kNN}(q, \{k_i\}), \quad o_{\text{mem}} = \text{softmax}(qK_{\mathcal{I}}^{\top})V_{\mathcal{I}}, \quad (34)$$

which is gated with ordinary attention. The store can span many batches and does not need gradients through nearest-neighbor selection.

Listing 16: Episodic KV retrieval; an ANN index replaces the dense distance

```
def memorizing_read(q, key_db, value_db, k):
    dist = torch.cdist(q.float(), key_db.float())
    idx = dist.topk(k, largest=False).indices
    keys, values = key_db[idx], value_db[idx]
    score = torch.einsum('...d,...kd->...k', q, keys)
    return torch.einsum('...k,...kd->...d', score.softmax(-1), values)
```



Evidence and failure modes. Experiments span language-model datasets and model scales up to billions of parameters, measuring perplexity versus memory size and retrieval configuration. The method demonstrates that latent episodic retrieval can improve a Transformer without differentiating through a huge store. Its risks are near-duplicate memorization, index cost, stale states after model updates, cross-example contamination, and weak source provenance. Similarity can retrieve the right surface continuation while failing compositional evidence use.

PRA relation and experiment. This is a close latent-memory precursor. The main difference is open kNN over anonymous stream states versus two-stage object identity and within-object attention. Compare (a) global episodic kNN, (b) exact URI restriction, (c) semantic object routing then token kNN, and (d) oracle object restriction. Hold total indexed KVs and returned keys fixed. Include near-duplicate continuation, multi-object composition, wrong URI, and stale-model cache conditions, and report source-level recall alongside perplexity.

6 Category IV: Retrieval-Augmented Pretraining and Generation

Retrieval augmentation places a corpus outside model parameters. The decisive axes are whether selection is supervised or latent, whether retrieved material enters as text or hidden state, and whether retrieval occurs once, per chunk, or per generated token.

6.1 kNN-LM

Motivation and mechanism. kNN-LM stores training-context representations paired with observed next tokens. At inference it builds a distance-weighted neighbor distribution and interpolates it with the parametric language model [Khandelwal et al., 2020]:

$$p_{\text{kNN}}(y | q) \propto \sum_{i: y_i=y} \exp(-d(q, k_i)/\tau), \quad (35)$$

$$p(y | q) = \lambda p_{\text{kNN}}(y | q) + (1 - \lambda) p_{\text{LM}}(y | q). \quad (36)$$

No retrieved passage is read compositionally; memory changes the final token distribution.

Listing 17: kNN-LM probability interpolation

```
def knn_lm(log_p_lm, q, keys, next_tokens, k, vocab, lam, tau):
    d, idx = torch.cdist(q, keys).topk(k, largest=False)
    weights = torch.softmax(-d / tau, dim=-1)
    p_knn = torch.zeros(*q.shape[:-1], vocab, device=q.device)
    p_knn.scatter_add_(-1, next_tokens[idx], weights)
    return torch.logaddexp(log_p_lm + math.log1p(-lam),
                          p_knn.clamp_min(1e-12).log() + math.log(lam))
```



Evidence, lineage, and limitations. The original work reports perplexity on WikiText-103 and domain-adaptation settings, comparing the base LM and cache/retrieval baselines. Improvements without parameter updates motivate later datastore adaptation, pruning, and efficient kNN work. Storage and ANN latency can be large; interpolation needs calibration; privacy risks follow from training-example memorization; and the mechanism is strongest for continuation similarity rather than multi-fact reasoning.

PRA relation and comparison. kNN-LM fuses at the output distribution; PRA fuses rich memory within hidden computation. Compare them with the same corpus encoder and retrieval budget on near-duplicate prediction versus questions requiring composition. Report perplexity, calibration, exact-match, datastore bytes, and latency. Add shuffled token labels for kNN-LM and shuffled objects for PRA. If PRA only matches kNN-LM on near-copy tasks at higher cost, its richer transport is not justified.

6.2 REALM

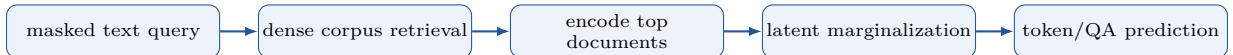
Motivation and mechanism. REALM jointly pretrains a dense retriever and a masked language model over an external corpus [Guu et al., 2020]. With latent document z ,

$$p(y | x) = \sum_{z \in \text{TopK}(x)} p_{\eta}(z | x) p_{\theta}(y | x, z), \quad (37)$$

where approximate maximum inner-product search supplies candidates and the language loss provides a learning signal to the retriever. Retrieved text is encoded jointly with the masked input, so the knowledge store can change without retraining all model parameters.

Listing 18: REALM-style latent-document marginalization

```
scores, doc_ids = retriever.topk(query_encoder(masked_input), k)
log_p_doc = scores.log_softmax(-1)
log_p_y = torch.stack([reader(masked_input, corpus[i]) for i in doc_ids], -1)
loss = -torch.logsumexp(log_p_doc + log_p_y, dim=-1).mean()
```



Evidence and failure modes. REALM evaluates open-domain QA and knowledge-intensive pretraining, comparing retrieval-free language models and retrieval pipelines with exact match and retrieval diagnostics. Its important legacy is end-to-end retriever learning from the task objective. Index refresh is expensive, top- k truncation biases the latent sum, corpus snapshots become stale, and retrieval may learn shortcuts when answers are predictable locally.

PRA relation and experiment. PRA should borrow REALM’s trainable selection rather than rely on fixed summary cosine scores. REALM materializes retrieved text in an encoder; PRA can reuse per-object layer KVs and explicit handles. Cross oracle/learned retrieval with text/cross-attention transport on the same corpus. Match retrieved tokens and index queries; evaluate locally sufficient and reference-required subsets, shuffled documents, random handles, warm caches, and index refresh after source updates.

6.3 Retrieval-Augmented Generation (RAG)

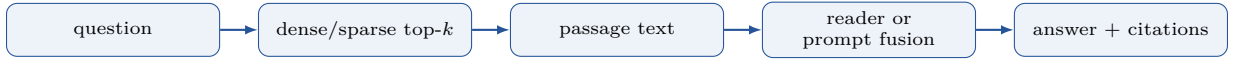
Motivation and mechanism. RAG combines a dense retriever with a seq2seq generator. RAG-Sequence holds a document mixture for a whole output; RAG-Token permits the latent document to vary by token [Lewis et al., 2020]:

$$p(y | x) = \prod_t \sum_{z \in \text{TopK}(x)} p_\eta(z | x) p_\theta(y_t | x, z, y_{<t}). \quad (38)$$

In common systems practice, retrieved passages are concatenated into a prompt rather than exactly marginalized, but the selection/materialization distinction is the same.

Listing 19: Minimal retrieve-then-read RAG pipeline

```
def rag_answer(question, retriever, generator, k):
    hits = retriever.search(question, k=k)
    context = "\n\n".join(f"[{h.id}] {h.text}" for h in hits)
    return generator(f"Sources:\n{context}\nQuestion: {question}\nAnswer:")
```



Evidence, descendants, and failures. RAG evaluates Natural Questions, TriviaQA, WebQuestions, CuratedTREC, MS MARCO-style knowledge tasks, and generation, comparing DPR, closed-book, and retrieval-augmented baselines with exact match and generation/diversity metrics. It seeded extensive work on reranking, query rewriting, corrective and agentic RAG. Failure modes include chunk boundary errors, retrieval miss, distractor sensitivity, prompt-token cost, citation mismatch, and stale indexes. A fluent answer does not prove that retrieved evidence caused the output.

PRA relation and direct comparison. Both keep knowledge external; they differ in timing and materialization. RAG usually chooses text before generation, while PRA can expose named latent objects by layer or step. Use one corpus, retriever, generator backbone, and evidence budget. Compare prompt text, encoder cross-attention, cached object KVs, and iterative retrieval. Report task quality, citation precision/recall, valid/disabled/shuffled effects, cold/warm latency, tokenization/encoding cost, and behavior as queries over the same documents repeat.

6.4 Atlas

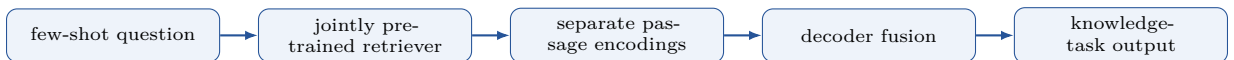
Motivation and mechanism. Atlas asks whether retrieval-augmented pretraining can make knowledge-intensive tasks data efficient, rather than treating retrieval only as an inference add-on [Izacard et al., 2022]. A dual-encoder retriever selects passages and a Fusion-in-Decoder reader encodes each question–passage pair separately before the decoder attends across all encoded passages. Retriever supervision can come from reader attention or leave-one-out evidence estimates. Abstractly,

$$z_{1:k} = \text{TopK}_{z \in \mathcal{D}} s_\eta(q, z), \quad p(y | q, z_{1:k}) = \text{FiD}_\theta(q, z_{1:k}). \quad (39)$$

The external index can be updated without storing all knowledge in model parameters, while joint pretraining teaches the retriever and reader a shared task interface.

Listing 20: Atlas-style retrieve, encode separately, and fuse in the decoder

```
def atlas_forward(question, retriever, reader, corpus, k):
    passages = corpus.fetch(retriever.topk(question, k))
    encoded = [reader.encode(question, passage) for passage in passages]
    return reader.decode(torch.cat(encoded, dim=1))
```



Evidence and limitations. Atlas evaluates few-shot and full-data learning on Natural Questions, KILT tasks, MMLU, and related knowledge-intensive benchmarks. The paper also varies index content and retrieval-training objectives. Its contribution is strongest at the training interface: retrieval provides updateable evidence and sample-efficient task adaptation. It still pays to retrieve and encode a passage set before generation, and reader-derived retriever targets can reinforce the reader’s own shortcuts.

PRA relation and experiment. Atlas supplies a stronger pretrained retrieval baseline than prompt-only RAG. A direct comparison should initialize both systems from the same corpus and language backbone, then vary the number of task examples. Atlas materializes passage encodings; PRA materializes selected per-layer chunk K/V. Match retrieved tokens, pretraining data, and online encoding cost. Oracle passages test transport, while valid-versus-shuffled evidence and index-refresh tests measure content and updateability.

6.5 Self-RAG

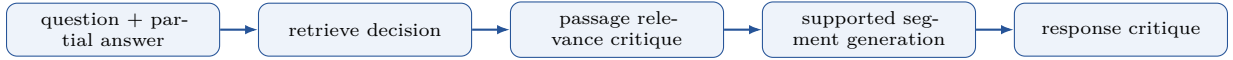
Motivation and mechanism. Fixed- k RAG retrieves even when parametric knowledge is sufficient and may accept irrelevant evidence without an explicit check. Self-RAG trains the generator to emit reflection tokens that control retrieval and assess passage relevance, support, and response quality [Asai et al., 2024]. A simplified policy alternates

$$a_t \in \{\text{RETRIEVE}, \text{CONTINUE}\}, \quad r_t = \text{critique}(q, z_t, y_{<t}), \quad (40)$$

then uses critique scores to filter or rank candidate continuations. Retrieval triggering and evidence criticism therefore become learned generation decisions rather than fixed pipeline settings.

Listing 21: Simplified adaptive retrieval and reflection loop

```
while not finished:
    action = model.next_control_token(question, draft)
    if action == RETRIEVE:
        passages = retriever.search(question + draft)
        passages = rank_by_reflection(model, question, draft, passages)
        draft += model.generate_segment(question, draft, passages)
    return draft
```



Evidence and limitations. Self-RAG evaluates open-domain QA, reasoning, fact verification, long-form factuality, and citation behavior with 7B- and 13B-scale models. Reflection tokens make retrieval frequency controllable, but their labels depend on a teacher and critic pipeline. A plausible self-critique can share the generator’s errors, and discrete retrieve/generate cycles still incur text and serial decoding cost.

PRA relation and experiment. Self-RAG is the closest retrieval analogue to PRA’s proposed progressive trigger. Compare a reflection-token trigger, a latent PRA gate, and a shared oracle trigger while holding the retriever and evidence fixed. Measure trigger precision/recall, unnecessary accesses, evidence support, serial latency, and content counterfactuals. A hybrid can expose PRA route confidence and provenance to a Self-RAG critic, but the critic’s approval must not replace removal and substitution tests.

6.6 Iterative Retrieval and Associative Closure

One-shot RAG ranks from the original question. Multi-hop tasks may instead reveal the next query only after an intermediate passage introduces an entity or relation. IRCOT makes this dependency explicit by alternating textual reasoning and retrieval [Trivedi et al., 2023]; Self-Ask decomposes a question into follow-ups [Press et al., 2022]. The shared computation is

$$z_h = R(q, s_{h-1}), \quad s_h = U(s_{h-1}, z_h), \quad qquadh = 1, \dots, H, \quad (41)$$

with a hop budget and stopping rule.

The important axis is not simply “one-shot versus agent.” Iteration can happen through generated subquestions, latent state, graph traversal, or retrieved gists. It may preserve a path of evidence identities or collapse every hit into an unordered pool. A fair comparison fixes final unique evidence and total retrieval work, then measures all-required-evidence recall, path validity, duplicate visits, stopping, latency, and answer quality. PRA’s proposed associative closure is the latent-gist member of this family; it remains proposed until tested against these established textual and structural loops.

6.7 RETRO

Motivation and mechanism. RETRO retrieves neighbors for fixed-size chunks from a massive token database, encodes each neighbor and its continuation, and interleaves chunked cross-attention in the decoder [Borgeaud et al., 2022]. For input chunk x_c and neighbors N_c ,

$$N_c = \text{kNN}(E(x_c), \mathcal{D}), \quad H'_c = H_c + \text{CrossAttn}(H_c, \text{Enc}(N_c)). \quad (42)$$

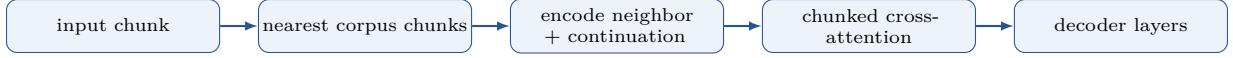
Retrieval happens at chunk boundaries, bringing non-parametric data closer to layer computation than prompt-only RAG.

Listing 22: RETRO-style chunk retrieval and cross-attention

```

for chunk in split(tokens, chunk_len):
    ids = index.search(chunk_encoder(chunk), k=neighbors)
    neighbor_states = encoder(corpus.with_continuations(ids))
    h = decoder_chunk(chunk, retrieved=neighbor_states) # selected layers cross-attend

```



Evidence, ablations, and limitations. RETRO trains models with a database on the order of trillions of tokens and compares perplexity and downstream behavior against larger parametric Transformers. Retrieval neighbors, database scale, cross-attention placement, and corpus overlap are central ablations. Results support non-parametric scaling, but the system inherits corpus/index cost, possible benchmark leakage, chunk-boundary mismatch, and retrieval latency; source identity is incidental rather than an explicit user-facing contract.

PRA relation and experiment. RETRO is one of PRA’s closest architectural ancestors: both transport external representations through cross-attention. PRA adds explicit handles, object boundaries, layer-specific reusable cache identity, progressive admission, and causal reference controls. Under one corpus and backbone, compare no handle, noisy handle, exact handle, and global semantic retrieval; keep returned tokens and PRA layers fixed. Measure selection recall, quality, per-stage latency, cache reuse, source corruption, and multi-object composition. Oracle selection isolates whether PRA’s transport works before router claims.

7 Category V: Attention-Level Retrieval and KV Runtime Management

This category is easy to conflate because every method manipulates latent states. The systems nevertheless perform different operations. Unlimiformer and Landmark Attention select semantic states or blocks. PagedAttention virtualizes physical allocation. StreamingLLM, H₂O, Scissorhands, SnapKV, FastGen, PyramidKV, and Quest reduce which already-produced states are loaded or retained. MiniCache compresses across layers, while CacheBlend repairs reusable chunk caches that were encoded outside their new prompt position. Selection, allocation, retention, representation compression, and reuse should therefore be measured separately.

7.1 Unlimiformer

Motivation and mechanism. Unlimiformer lets a pretrained encoder-decoder process arbitrarily segmented input, indexes all encoder hidden states, and replaces dense decoder cross-attention with exact attention over MIPS-selected states [Bertsch et al., 2023]. Because the cross-attention score is an inner product, each decoder query can retrieve its largest candidate logits without first forming the full query–key matrix:

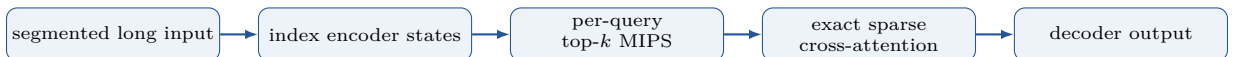
$$\mathcal{I}(q) = \text{TopK}_i(q^\top k_i), \quad o = \text{softmax}(qK_{\mathcal{I}}^\top)V_{\mathcal{I}}. \quad (43)$$

Listing 23: Token-state MIPS followed by exact attention

```

def unlimiformer_cross_attention(q, index, value_store, k):
    scores, ids = index.search(q, k)
    values = value_store[ids]
    return torch.einsum('...k,...kd->...d', scores.softmax(-1), values)

```



Evidence and limitations. The paper evaluates long-input encoder-decoder tasks, especially summarization, against truncated, segmented, and long-context baselines using ROUGE and memory/length measures. It also studies training-free wrapping and fine-tuning. Selection occurs at token granularity and can miss jointly useful low-ranked tokens; the index is tied to the current encoded input; ANN setup and query cost remain; and source provenance must be reconstructed from token offsets.

PRA relation and comparison. Unlimiformer and PRA meet at sparse latent attention. Unlimiformer indexes all token states of the current long input, while PRA first selects named objects and may persist their per-layer KVs. Compare full cross-attention, Unlimiformer token MIPS, PRA object routing plus dense within-object attention, and a hierarchical object-to-token cascade. Match returned KVs, encoder FLOPs, and peak bytes; sweep object size, distractors, repeated queries, and object-level corruption. The cascade tests whether explicit identity improves efficiency without sacrificing token precision.

7.2 Landmark Attention

Motivation and mechanism. Ordinary sparse attention requires a fixed graph or an external retriever. Landmark Attention instead trains the model’s own attention to use one special landmark token as the representative key for each token block [Mohtashami and Jaggi, 2023]. At inference, a query first scores landmarks, retrieves the top blocks, and then attends to token K/V inside those blocks. If λ_b is the landmark for block b ,

$$\mathcal{B}(q) = \text{TopK}_b(q^\top k_{\lambda_b}), \quad o = \text{Attn}(q, K_{\mathcal{B}(q)}, V_{\mathcal{B}(q)}). \quad (44)$$

The method preserves random access to distant blocks while avoiding attention over every token. Its optional context-miss token also suggests a learned decision not to retrieve.

Listing 24: Landmark routing followed by block attention

```
def landmark_attention(query, landmark_keys, block_keys, block_values, k):
    block_ids = (query @ landmark_keys.T).topk(k).indices
    keys = torch.cat([block_keys[i] for i in block_ids], dim=-2)
    values = torch.cat([block_values[i] for i in block_ids], dim=-2)
    return scaled_dot_product_attention(query, keys, values)
```



Evidence and limitations. The paper evaluates PG19 language modeling and passkey retrieval, including a LLaMA-7B adaptation beyond its original context. It studies block retrieval granularity and a context-miss threshold. Landmark keys are learned in the same attention space as their consumers, but every block must be ingested and assigned a landmark. Blocks remain positional rather than stable versioned objects, and a wrong landmark decision hides every token inside the missed block.

PRA relation and experiment. Landmark Attention is the closest token-stream analogue to PRA’s corrected chunk gists. Both rank a small projected-key representative and transport detailed token K/V after selection. The differences are identity, lifecycle, and scope: landmarks index blocks of an ingested stream, whereas PRA gists belong to resolved URI objects that may be cached and invalidated. Compare both with identical chunk boundaries, top- k , and attention projections; then vary object reuse, reorder blocks, change versions, and measure cold versus warm construction. This experiment can reveal whether explicit object identity adds value beyond learned block routing.

7.3 PagedAttention

Motivation and mechanism. PagedAttention is a serving-memory design in vLLM, not a semantic attention rule. It partitions a sequence’s KV cache into fixed-size physical blocks addressed through a logical block table, reducing fragmentation and enabling copy-on-write sharing [Kwon et al., 2023]. For logical token position t ,

$$b = \lfloor t/B \rfloor, \quad o = t \bmod B, \quad (K_t, V_t) = \text{blockTable}[b][o]. \quad (45)$$

Listing 25: Logical-to-physical KV lookup

```
def paged_lookup(logical_positions, block_table, blocks, block_size):
    logical_block = logical_positions // block_size
    offset = logical_positions % block_size
    physical_block = block_table[logical_block]
    return blocks[physical_block, offset]
```



Evidence, systems lineage, and failures. vLLM evaluates serving throughput, latency, memory utilization, and scheduling across model sizes and workloads against contemporary inference systems. Block size, batching, sharing, and scheduling are systems variables rather than accuracy ablations. Paging cannot decide which knowledge is useful; small blocks increase metadata, large blocks reintroduce internal fragmentation, and shared physical storage must preserve isolation and copy-on-write semantics.

PRA relation and experiment. The designs compose: PRA supplies semantic object identity and PagedAttention supplies allocation. Map an immutable (u, v, ℓ, m, p) object to reference-counted KV blocks and maintain per-sequence block tables/masks. Compare contiguous and paged PRA caches under cold, warm, and authorized cross-request reuse. Measure bytes allocated, fragmentation, blocks shared, avoided encoding, time to first token, throughput, eviction, and a confidentiality test where another sequence’s private cache must leave the target logits invariant.

7.4 StreamingLLM

Motivation and mechanism. StreamingLLM observes that autoregressive models often assign disproportionate attention to initial “sink” tokens even when those tokens are not semantically important. Retaining a few initial KVs plus a rolling recent window allows stable streaming beyond the training context [Xiao et al., 2023]:

$$\mathcal{K}_t = \{0, \dots, s-1\} \cup \{t-w+1, \dots, t\}. \quad (46)$$

Middle-history KVs are evicted; no semantic retrieval or reconstruction is attempted.

Listing 26: Attention-sink plus rolling-window retention

```
def streaming_keep(length, sinks=4, window=1024):
    first = torch.arange(min(sinks, length))
    recent = torch.arange(max(sinks, length-window), length)
    return torch.unique(torch.cat([first, recent]))
```



Evidence and failure modes. The paper tests long streaming language modeling and generation with LLaMA-style models, comparing sliding windows and cache baselines through perplexity and long-run behavior. Sink tokens repair an attention-distribution pathology; they do not preserve arbitrary facts. Middle-history details are irrecoverable, sink behavior depends on model training, and the method is not a solution for non-chronological external knowledge.

PRA relation and experiment. StreamingLLM manages the local chronological cache; PRA retrieves named backing objects. Combine them by keeping sinks/recent discourse locally and dereferencing remote evidence. Under a fixed total KV budget, compare rolling window, sink+window, PRA, and the hybrid. Sweep query distance, explicit handles, and whether lost facts are still present in the backing store. Charge PRA faults and test shuffled content; test StreamingLLM with sink removal and exact middle-history probes.

7.5 H₂O: Heavy-Hitter Oracle

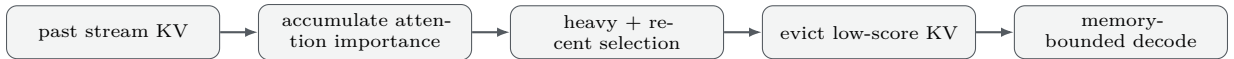
Motivation and mechanism. H₂O treats KV eviction as a dynamic submodular selection problem. It retains recent tokens and “heavy hitters” with large accumulated attention contribution [Zhang et al., 2023]. A simplified score is

$$I_i^{(t)} = I_i^{(t-1)} + \sum_h A_{h,t,i}, \quad \mathcal{K}_t = \text{Recent}_r \cup \text{TopK}_h(I^{(t)}). \quad (47)$$

The policy reacts to observed decoder attention rather than using a static window.

Listing 27: Educational heavy-hitter update

```
def h2o_keep(importance, attn_to_past, recent_ids, heavy_budget):
    importance[attn_to_past.numel()] += attn_to_past.detach()
    heavy = importance.topk(heavy_budget).indices
    return torch.unique(torch.cat([heavy, recent_ids])), importance
```



Evidence and limitations. H₂O evaluates OPT, LLaMA, and GPT-NeoX across language and generation tasks, measuring quality preservation, cache size, throughput, and latency against inference systems and eviction baselines. Its reported systems gains are conditional on model, hardware, batch, cache fraction, and implementation. Accumulated attention is a lagging proxy, early errors are irreversible, low-attention setup facts can matter later, and raw attention mass is not causal importance.

PRA relation and experiment. H₂O decides which states from one active stream remain; PRA decides which external objects enter. Use H₂O within each selected PRA object or across the resident object working set, but preserve URI/version metadata for every KV. Compare full object KVs, H₂O-compressed objects, object-level eviction, and both tiers. Sweep delayed-use facts, distractors, object count, and warm-cache reuse. Removal/substitution tests should verify that retained heavy hitters, rather than generic branch activation, cause correct answers.

7.6 Scissorhands

Motivation and mechanism. Scissorhands builds on the “persistence of importance” hypothesis: tokens important in earlier attention tend to remain important later. It maintains importance statistics and discards KVs predicted to stay unimportant [Liu et al., 2023]. At an eviction step,

$$\mathcal{K} = \text{Recent}_r \cup \text{TopK}_{B-r}(\text{score}_{\text{history}}(i)). \quad (48)$$

Policies can be applied per head/layer and periodically rather than at every token.

Listing 28: Periodic history-score pruning

```
def scissorhands(kv, score, recent, budget):
    protected = torch.arange(max(0, len(score)-recent), len(score))
    old = torch.arange(0, max(0, len(score)-recent))
    keep_old = old[score[old].topk(max(0, budget-len(protected))).indices]
    keep = torch.cat([keep_old, protected])
    return kv.index_select(-2, keep), score.index_select(0, keep)
```



Evidence and failure modes. The work evaluates cache reduction across language models/tasks using perplexity or task quality plus memory and throughput. Comparisons with window and heuristic eviction establish the value of history-aware scores. Importance persistence can fail under topic shifts or late references; dropped tokens cannot be reconstructed; per-head policies complicate kernels; and attention history remains a correlational diagnostic.

PRA relation and experiment. Scissorhands compresses already materialized stream history, whereas PRA can reload a stable object after eviction. Compare irreversible token eviction, eviction plus raw backing-text reload, and eviction plus cached PRA object KVs. Match total bytes and include late queries after long distractor gaps. Report reload rate, quality, end-to-end latency, and counterfactual content effects. Object boundaries may provide a cleaner eviction unit than anonymous tokens, but that is a testable hypothesis.

7.7 SnapKV

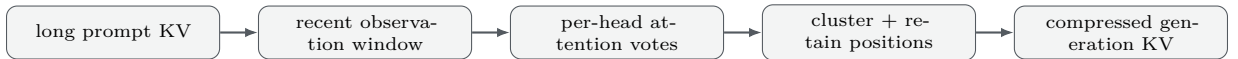
Motivation and mechanism. SnapKV compresses a long prompt’s KV cache before generation by using attention from a recent observation window to identify prompt positions likely to matter during decoding, then pooling nearby selections [Li et al., 2024]. A schematic per-head rule is

$$s_i = \text{pool} \left(\sum_{q \in W_{\text{obs}}} A_{q,i} \right), \quad \mathcal{K} = \text{TopK}_B(s). \quad (49)$$

Unlike online eviction, it takes a “snapshot” of prompt relevance before the answer grows.

Listing 29: Observation-window prompt selection

```
def snapkv_scores(attn, observation, pool=7):
    # attn: [heads, query, key]; exclude generated/recent keys as appropriate
    score = attn[:, -observation:, :].sum(dim=1).unsqueeze(1)
    return F.max_pool1d(score, kernel_size=pool, stride=1, padding=pool//2).squeeze(1)
```



Evidence and limitations. SnapKV evaluates long-context QA/retrieval and generation across instruction-tuned LLMs, measuring task quality, prompt-cache reduction, and speed against eviction/compression baselines. Its main assumption is that the observation window predicts future importance. Later reasoning can change the relevant region; prompt positions lose object/version identity; selection errors are irreversible unless the source remains elsewhere; and head-wise layouts complicate reuse.

PRA relation and experiment. SnapKV is query-aware token compression after eager prompt ingestion; PRA is object admission before or during latent use. Compare full prompt, SnapKV, PRA, and PRA with SnapKV-compressed object KVs. Match resident bytes and include questions whose relevance is evident initially versus revealed after intermediate reasoning. Measure prefill, selection, cold encoding, decode latency, exact evidence retention, and valid/shuffled object effects. A multi-fidelity PRA cache could store a SnapKV snapshot and escalate to full object KVs on low confidence.

7.8 FastGen and PyramidKV: Head- and Layer-Adaptive Budgets

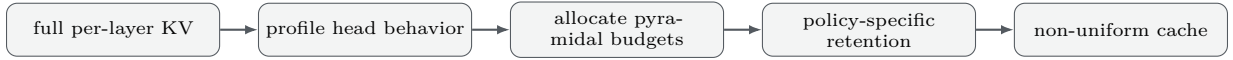
Motivation and mechanism. A uniform cache budget assumes that every attention head and layer uses history in the same way. FastGen profiles each head and assigns a retention policy that matches its observed structure: local heads keep a window, special-token heads keep designated positions, and broadly distributed heads retain more of the cache [Ge et al., 2023]. PyramidKV operates along a different axis. It reports that attention is broad in lower layers and more concentrated in higher layers, then uses a decreasing layer budget [Cai et al., 2024]. A generic allocation is

$$B_{\ell,h} = B_{\text{total}} \frac{w_{\ell,h}}{\sum_{j,g} w_{j,g}}, \quad (50)$$

where the weight $w_{\ell,h}$ comes from head profiling, layer depth, or both. The equation does not prescribe either paper’s exact policy; it exposes their shared principle that cache capacity should follow attention structure rather than tensor shape.

Listing 30: Composing head policy with a layer-dependent cache budget

```
for layer, heads in enumerate(model.attention_heads):
    layer_budget = pyramid_budget(layer, model.num_layers, total_budget)
    for head in heads:
        policy = profiled_policy(head) # local, special-token, or broad
        head.cache = policy.retain(head.cache, budget=layer_budget[head.id])
```



Evidence and limitations. FastGen evaluates training-free adaptive compression across language-model tasks and reports memory savings with small quality changes under its tested settings. PyramidKV evaluates LongBench and needle retrieval, including aggressive retention fractions. Both depend on attention patterns transferring across inputs. A head classified as local can become globally important on another task, and a depth trend does not identify which individual evidence tokens must survive. Specialized per-head layouts can also reduce kernel regularity.

PRA relation and experiment. These methods can compress each selected PRA object’s token K/V after object and chunk routing. Compare uniform and adaptive budgets under the same total bytes, with per-layer routing recall and content counterfactuals. The critical failure test delays a reference until a head or layer previously classified as low-value must retrieve it. A successful composition would reduce resident bytes without changing which URI and chunk caused the answer.

7.9 Quest: Query-Aware Page Selection

Motivation and mechanism. Retention policies decide what survives before the future query is known. Quest retains paged K/V metadata and decides which pages to load for the current query [Tang et al., 2024]. For each key dimension, a page stores extrema $k_p^{\min}$ and $k_p^{\max}$. The query signs choose an upper bound on the page’s possible inner product,

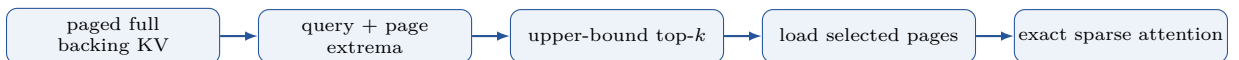
$$U(q, p) = \sum_j \max(q_j k_{p,j}^{\min}, q_j k_{p,j}^{\max}), \quad \mathcal{P}(q) = \text{TopK}_p U(q, p). \quad (51)$$

Only selected pages are transferred for exact attention. The method therefore reduces memory bandwidth without first discarding the full backing cache.

Listing 31: Quest-style query bound over KV pages

```
def quest_page_scores(query, page_key_min, page_key_max):
    lower = query * page_key_min
    upper = query * page_key_max
    return torch.maximum(lower, upper).sum(dim=-1)

pages = quest_page_scores(q, key_min, key_max).topk(page_budget).indices
output = exact_attention(q, load_keys(pages), load_values(pages))
```



Evidence and limitations. Quest reports self-attention speedups and end-to-end latency improvements on long-context tasks while preserving quality in the evaluated configurations. Its page bounds are inexpensive and query dependent, but loose bounds load irrelevant pages, tight budgets can omit diffuse evidence, and page layout determines selection granularity. The full cache still occupies a backing tier even when HBM traffic falls.

PRA relation and experiment. Quest performs physical page routing inside an already materialized context; PRA performs semantic object and chunk routing before transport. The two-stage composition is natural: PRA selects URI chunks and Quest selects pages inside their token K/V. Compare flat Quest over all candidate objects, hierarchical PRA-to-Quest, and PRA with dense selected chunks. Match page metadata, loaded bytes, and top- k tokens. Report both semantic recall and actual bandwidth, because a good router that loads loose pages may not improve the systems frontier.

7.10 MiniCache and CacheBlend: Depth Compression and Reusable Chunks

Motivation and mechanism. Token eviction compresses the sequence dimension, but KV redundancy also appears across depth and repeated prompts. MiniCache observes similarity between adjacent middle-to-late layers and merges corresponding states after separating vector direction from magnitude [Liu et al., 2024a]. Distinct state pairs can remain unmerged. CacheBlend addresses a different reuse problem: a document chunk encoded alone does not contain the cross-chunk interactions it would receive in a new RAG prompt. It reuses offline chunk K/V but selectively recomputes important tokens to repair the blended context [Yao et al., 2024].

Listing 32: Two orthogonal cache transformations

```
# Depth compression: share similar adjacent-layer states.
merged = merge_direction_preserve_norm(kv[layer], kv[layer + 1], keep_distinct=True)

# Reuse repair: blend cached chunks, then recompute selected contextual tokens.
prompt_kv = concatenate(cached_chunk_kv)
repair_ids = select_recompute_tokens(prompt_kv, query)
prompt_kv[repair_ids] = model.prefill_tokens(prompt, repair_ids)
```



Evidence and limitations. MiniCache evaluates several LLaMA, Mistral, Phi, and Mixtral variants across language and long-context benchmarks, reporting compression, throughput, and quality. CacheBlend evaluates RAG serving with several open models and datasets, reporting time to first token and throughput against full recomputation and prefix reuse. Cross-layer similarity is model dependent, while chunk reuse is position and context dependent. In both cases, approximate equivalence must be tested on compositional questions, not inferred from cache similarity alone.

PRA relation and experiment. MiniCache can reduce the depth cost of a PRA object; CacheBlend is a direct baseline for PRA’s reusable-object systems claim. Compare full per-request encoding, prefix-only reuse, CacheBlend repair, position-independent PRA cross-attention, and PRA with depth-compressed object K/V. Use reordered documents, multi-document composition, stale versions, and shared-cache workloads. Measure answer quality, repair fraction, encoding avoided, bytes transferred, time to first token, and isolation. PRA should not claim reusable K/V advantage unless it beats these stronger reuse baselines at matched quality.

7.11 Prompt Cache and KV Quantization

Prompt Cache names reusable prompt modules, precomputes their attention state, and uses a schema to preserve positional validity when modules recur across prompts [Gim et al., 2024]. It addresses avoided prefill for known repeated text, not semantic discovery. KIVI addresses a different resource axis: asymmetric low-bit quantization stores keys per channel and values per token to reduce cache bytes without changing which tokens are active [Liu et al., 2024c]. These methods make two orthogonal facts explicit:

$$\text{reuse controls recomputation, } q\text{quadquantization controls bytes per retained state.} \quad (52)$$

Both compose with semantic admission. A PRA object cache could use schema-like identity and positional provenance, then quantize inactive or cold native K/V. Evaluation must cross cache hit/miss with quantization level and selected fraction. Report avoided encoding, transfer bytes, dequantization cost, active K/V, and output fidelity; a high hit rate or compression ratio alone is not a quality–resource result.

7.12 KV management design map

The methods above can be compared by the unit they control and by whether omitted detail still exists in a backing tier. That distinction predicts recoverability: page selection can load another page on the next query, whereas destructive token eviction cannot recover a dropped state unless raw content is re-encoded.

Table 3: KV-management mechanisms control different logical and physical units.

Method	Controlled unit	Decision time	Backing detail	Training change	Principal risk
PagedAttention	physical block	allocation	full K/V remains	no	fragmentation or isolation bug
StreamingLLM	token positions	decode	evicted middle absent	no	unrecoverable old detail
H ₂ O	token positions	decode	evicted tokens absent	no	lagging attention score
Scissorhands	token/head K/V	periodic decode	evicted tokens absent	no	importance changes later
SnapKV	prompt token/head	before generation	source may remain external	no	observation window predicts poorly
FastGen	head policy	profile/prefill	policy dependent	no fine-tuning	profile fails to transfer
PyramidKV	layer budget	prefill	omitted tokens absent	no	depth heuristic misses evidence
Quest	K/V page	each query	full backing K/V remains	no	loose bounds or page miss
MiniCache	adjacent layers	prefill/decode	selected exceptions remain	no	cross-layer mismatch
CacheBlend	reusable text chunk	prefill	raw text and offline K/V remain	no	insufficient contextual repair
Prompt Cache	named prompt module	before requests	reusable attention state	no	position/schema mismatch
KIVI	numeric K/V elements	cache write/read	quantized full cache	no	quantization error or kernel cost
Landmark Attention	token block	each query	block K/V remains	yes	wrong landmark hides block
PRA	URI chunk, then token K/V	layer/step	raw/versioned object may remain	yes/runtime policy	wrong route, stale identity

The map exposes three promising compositions. PagedAttention can allocate any of the logical caches. Quest can reduce bandwidth inside a selected Landmark or PRA block. MiniCache or an adaptive budget can compress the resulting resident tensors. CacheBlend, by contrast, is a direct alternative when the goal is reuse of text chunks inside ordinary self-attention. End-to-end comparisons must include all metadata, repair, transfer, and re-encoding costs; nominal cache bytes alone do not identify the faster system.

8 Category VI: Structural and Hierarchical Indexing

Flat chunk retrieval treats document structure as noise. Hierarchical and graph approaches make sentences, sections, entities, communities, or claims explicit intermediate units. Their advantage is not merely fewer tokens: the index supplies a reasoning topology and a natural provenance coordinate system.

8.1 Hierarchical Attention Networks (HAN)

Motivation and mechanism. HAN mirrors document composition: a bidirectional word encoder and attention pool create sentence vectors, then a sentence encoder and attention pool create a document vector [Yang et al., 2016]. At either level,

$$u_i = \tanh(Wh_i + b), \quad \alpha_i = \frac{\exp(u_i^\top u_c)}{\sum_j \exp(u_j^\top u_c)}, \quad v = \sum_i \alpha_i h_i. \quad (53)$$

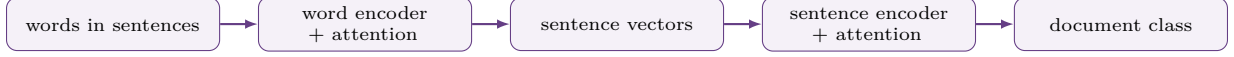
The context vector u_c learns which units matter for classification. Hierarchy reduces a variable document to semantically meaningful intermediate representations.

Listing 33: Reusable hierarchical attention pool

```

class AttnPool(nn.Module):
    def __init__(self, d):
        super().__init__(); self.proj = nn.Linear(d, d); self.context = nn.Parameter(torch.randn(d))
    def forward(self, h, mask=None):
        score = torch.tanh(self.proj(h)) @ self.context
        if mask is not None: score = score.masked_fill(~mask, -torch.inf)
        return torch.einsum('bt,btd->bd', score.softmax(-1), h)

```



Evidence and limitations. The original paper evaluates document classification on Yelp, IMDB, Yahoo Answers, and Amazon review datasets against bag-of-words, convolutional, and recurrent baselines using classification accuracy. Word- and sentence-level attention visualizations are useful diagnostics, not causal explanations. The hierarchy is fixed at two levels, pooling is lossy, bidirectional encoding is not a streaming solution, and the model is designed for one document-level prediction rather than open-ended retrieval.

PRA relation and experiment. HAN supplies the principle that intermediate document units deserve representations. PRA generalizes those units to named sections, tables, records, and code symbols and permits later expansion. Compare flat chunks, HAN-style sentence summaries, and URI hierarchy on classification plus evidence-required QA. Remove or substitute high-attention sentences and selected PRA nodes. A strong hybrid uses HAN pooling to construct cheap summaries while preserving children as resolvable objects.

8.2 RAPTOR

Motivation and mechanism. RAPTOR recursively embeds and clusters leaf chunks, asks a language model to summarize each cluster, and repeats until a tree of increasingly abstract nodes is formed [Sarathi et al., 2024]. With nodes V_0 at the leaves,

$$V_{\ell+1} = \{\text{summarize}(C) : C \in \text{cluster}(\{E(v) : v \in V_\ell\})\}. \quad (54)$$

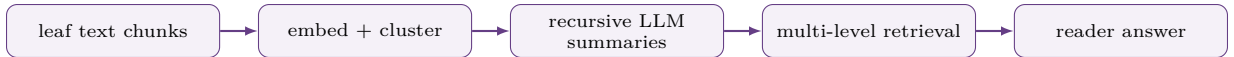
Retrieval can search a collapsed set of all levels or traverse the tree, letting a query choose between detailed evidence and broad synthesis.

Listing 34: Bottom-up RAPTOR index construction

```

level = chunk(document)
tree = [level]
while len(level) > 1:
    groups = cluster(embed(level))
    level = [summarizer(group) for group in groups]
    tree.append(level)
return tree

```



Evidence, paper trail, and failures. RAPTOR reports QA results on QuALITY, NarrativeQA, and QASPER, comparing conventional retrieval and long-context/retrieval baselines with accuracy or task metrics. The paper highlights a 20-point absolute QuALITY gain for its GPT-4 configuration; that number is conditional on its model, tree, prompts, and evaluation. Descendants study dynamic maintenance and cheaper tree construction. Indexing is expensive, abstractive summaries can omit or invent facts, clustering is unstable, and an error at a high node can misroute an entire subtree.

PRA relation and experiment. RAPTOR chooses representations during indexing; PRA chooses when and at which layer a representation becomes resident. Assign immutable URIs to every tree node and let the router escalate from summary to children. Compare flat RAG, collapsed-tree retrieval, explicit tree traversal, and PRA transport using the same tree. Measure answer quality, retrieved source tokens, LLM index cost, expansions, cache reuse, summary corruption, and whether leaf evidence supports generated claims.

8.3 MemWalker

Motivation and mechanism. MemWalker recursively summarizes a long document into a memory tree and has an LLM navigate it by choosing children until sufficient evidence is found [Chen et al., 2023a]. At node v_t , a policy reads the query and child summaries:

$$a_t \sim \pi_\theta(a \mid q, s(v_t), \{s(c) : c \in \text{child}(v_t)\}), \quad v_{t+1} = \text{child}(v_t, a_t). \quad (55)$$

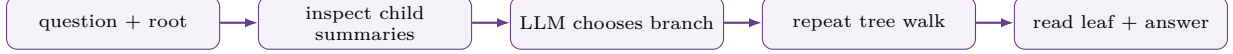
The traversal trace externalizes intermediate access decisions instead of retrieving all nodes in one ranking call.

Listing 35: Bounded tree walk

```

node = root
for depth in range(max_depth):
    action = navigator.choose(question, node.summary,
                             [c.summary for c in node.children])
    if action == "answer": break
    node = node.children[action]
return reader(question, node.source_text)

```



Evidence and failure modes. MemWalker evaluates long-context reasoning/QA against retrieval and long-context baselines, measuring answer quality and traversal behavior. It is closer to an agent than static RAG, but the action space is confined to a prebuilt tree. Serial LLM calls add latency, greedy early choices can make relevant siblings unreachable, summary errors compound, and navigation traces show decisions without proving causal use of the selected leaf.

PRA relation and experiment. Both implement progressive disclosure. MemWalker serializes branch decisions as language-model actions; PRA proposes a latent router inside selected layers. Use the same tree and compare textual navigation, latent top- k child selection, oracle paths, and a hybrid that escalates ambiguous nodes to the agent. Match node-expansion budgets and report latency, depth, backtracking, path recall, valid/shuffled leaf effects, and calibration at the escalation boundary.

8.4 PageIndex

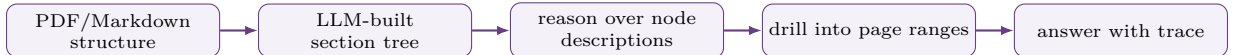
Motivation and mechanism. PageIndex builds a table-of-contents-like tree from natural document sections and uses an LLM to reason over titles, summaries, and page ranges rather than relying on vector similarity [Zhang et al., 2025]. A typical node is (id, title, $[p_s, p_e]$, summary, children); retrieval is a reasoned tree search followed by page/section extraction.

Listing 36: Reasoning-first section traversal

```

frontier = [root]
while budget.remaining() and not evidence_sufficient(frontier, query):
    decision = llm_rank(query, describe(frontier))
    frontier = expand_selected(frontier, decision.node_ids)
pages = merge_ranges([n.page_range for n in frontier])
return answer(query, pdf.extract(pages))

```



Evidence status and reproducibility. PageIndex is an evolving open-source/product system, not a settled peer-reviewed architecture result. This survey pins its inspection target to repository commit [7592163e2a37](#) (mirror snapshot synchronized 15 June 2026) and records the repository’s then-documented default `gpt-4o-2024-11-20`; access date is 3 August 2026. The repository advertises 98.7% FinanceBench accuracy, but this remains a vendor-reported compound-system result. Reproduction must freeze PDFs/questions, parser/OCR, tree-building and answering prompts, model versions, judge, exact-match policy, retries, and dollar/token cost. Natural structure can itself be missing or malformed, and every LLM tree call can introduce nondeterminism.

PRA relation and experiment. PageIndex supplies a strong logical namespace for PRA: node IDs/page ranges become versioned URIs, and selected sections can materialize as text or per-layer KV. Compare vector RAG, PageIndex traversal, PRA latent traversal, and agent escalation on the same tree. Stratify exact lookup, cross-reference, multi-section synthesis, and cross-document reasoning. Report source-page recall, answer/citation quality, tree calls, latency, cost, and invalidation after a PDF revision.

8.5 GraphRAG

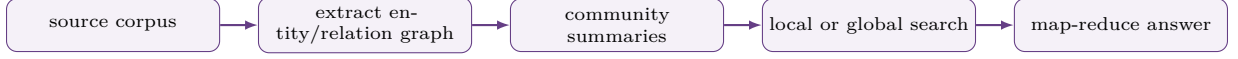
Motivation and mechanism. GraphRAG extracts entities, relations, and claims from a corpus, clusters the resulting graph into communities, and creates community reports [Edge et al., 2024]. Local search expands around query entities; global search maps a question over community reports and reduces partial answers:

$$\hat{y} = \text{reduce}\{\text{answer}(q, R_c) : c \in \text{select}(q, \mathcal{C})\}. \quad (56)$$

The graph exposes corpus-wide themes and multi-hop relationships that may not be similar to the surface query.

Listing 37: GraphRAG global map-reduce sketch

```
reports = select_community_reports(query, graph.communities, token_budget)
partials = [llm.map_answer(query, report) for report in reports]
return llm.reduce_answer(query, partials)
```



Evidence and failure modes. The GraphRAG preprint studies global sensemaking over roughly million-token corpora and reports LLM-judged comprehensiveness and diversity relative to conventional RAG. These are inference-time judgments over an expensive LLM-derived index, so model, prompts, judge, and cost are integral conditions. Extraction errors, entity resolution, hallucinated relations, index cost, stale derived summaries, and judge bias are prominent risks. Local factual QA and global thematic synthesis are distinct tasks.

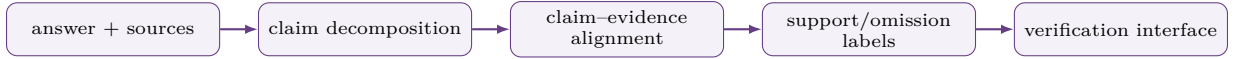
PRA relation and experiment. A PRA URI can address a node, neighborhood, graph query result, or community report. The hard problem becomes dependency-aware invalidation: changing one source may stale many derived KVs. Compare flat RAG, GraphRAG text transport, and graph-node PRA transport on local and global questions. Match source and answer models; report index cost separately from per-query cost, provenance to original spans, stale-update tests, and counterfactual removal of decisive nodes.

8.6 PaperTrail as provenance and HCI

Motivation and mechanism. PaperTrail is not a context architecture. It decomposes an answer and its sources into claims and evidence, then visualizes supported claims, unsupported assertions, and omitted source information [Martin-Boyle et al., 2026]. If C is answer claims and E source evidence units, the interface exposes a relation $R \subseteq C \times E$ plus unsupported $C \setminus \text{dom}(R)$ and uncovered source claims.

Listing 38: Claim-evidence audit pipeline

```
answer_claims = decompose_claims(answer)
source_claims = decompose_claims(sources)
links = align_and_verify(answer_claims, source_claims)
render(answer_claims, source_claims, links,
        unsupported=set(answer_claims)-links.claims())
```



Evidence and limitations. The 2026 work evaluates PaperTrail in a within-subjects study with 26 researchers performing scholarly editing tasks. The interface lowered trust relative to a baseline but did not produce corresponding behavioral change: participants still relied on generated edits when verification was cognitively costly. This is an HCI result about provenance presentation, not evidence for longer context or retrieval quality. Claim decomposition and alignment can themselves be wrong, and more visible evidence can overload rather than assist a user.

PRA relation and experiment. PRA’s URI/version and routing traces can populate a PaperTrail-like view, but attended memory is not causal support. Record selected objects, source spans, cache versions, generated claims, and outputs under removal/substitution. Compare citation-only, route-trace, and counterfactual claim-evidence interfaces. Measure verification accuracy, time, calibrated trust, unsupported-claim detection, omissions, and cognitive load. This positions PaperTrail correctly as provenance/HCI rather than a competitor in the context architecture table.

9 Category VII: Agents with Search and Tool Use

Agents move relevance decisions outside a single forward pass. They can discover unknown sources, reformulate a question after observing evidence, act on the world, and backtrack. Their generality costs serial latency, tokens, tool failures, and a larger audit surface. PRA should be compared as a restricted fast path for repeated read-only access, not as a replacement for open-ended action.

9.1 ReAct

Motivation and mechanism. ReAct interleaves natural-language reasoning with actions and observations, allowing evidence acquisition to change the next reasoning step [Yao et al., 2023]. A trajectory is

$$\tau = (q, r_1, a_1, o_1, r_2, a_2, o_2, \dots, y), \quad (r_t, a_t) \sim \pi_\theta(\cdot \mid q, \tau_{<t}). \quad (57)$$

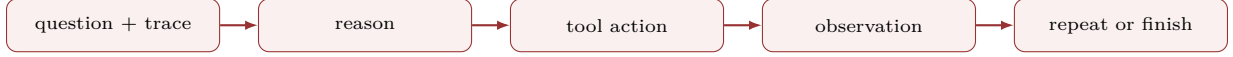
Reasoning maintains a task state; tools ground or alter that state; the policy decides when to stop. Search is only one environment—the paper also studies interactive tasks.

Listing 39: ReAct execution loop with an explicit budget

```

trace = []
for step in range(max_steps):
    thought, action = model.next(question, trace)
    if action.name == "finish": return action.answer, trace
    observation = tools[action.name](**action.arguments)
    trace.append((thought, action, observation))
raise BudgetExceeded(trace)

```



Evidence, descendants, and failures. ReAct evaluates HotpotQA and FEVER plus ALFWorld and WebShop, comparing standard prompting, chain-of-thought, act-only, and ReAct using exact match, factual accuracy, or environment success. Prompt examples, base model, tool interface, and action budget materially condition results. Descendants add planning, reflection, memory, and richer tools. Common failures are reasoning loops, malformed calls, irrelevant searches, premature stopping, prompt injection in observations, and high latency.

PRA relation and experiment. Both progressively acquire context. ReAct remains necessary when sources are unknown, actions have side effects, or explicit planning and backtracking are essential. Compare ReAct search, PRA over a known reference graph, and a tiered policy that tries local context, cached URI memory, structural navigation, then search. Match evidence/action budgets and report answer quality, calls, tokens, latency, provenance, cache reuse, and escalation calibration. Cache successful read-only discoveries as versioned references, never silently replay side-effecting actions.

9.2 Self-Ask with Search

Motivation and mechanism. Self-Ask addresses compositionality by explicitly generating follow-up questions. An external search component answers each subquestion, and the model combines intermediate answers into the final response [Press et al., 2022]:

$$q_t = \text{decompose}(q, a_{<t}), \quad e_t = \text{search}(q_t), \quad y = \text{compose}(q, \{q_t, e_t\}). \quad (58)$$

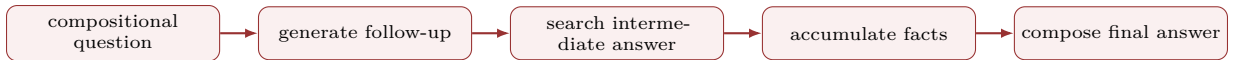
The textual protocol makes the retrieval chain easy to inspect and separates decomposition errors from search errors.

Listing 40: Self-Ask protocol

```

facts = []
while model.needs_followup(question, facts):
    subquestion = model.followup(question, facts)
    evidence = search(subquestion)
    facts.append((subquestion, evidence))
return model.final_answer(question, facts)

```



Evidence and limitations. The work studies multi-hop/compositional question answering and the “compositionality gap”, comparing direct answer, chain-of-thought, and search-assisted decomposition with exact match. Decomposition improves observability, but one wrong subquestion can derail all later hops; intermediate search snippets may be noisy; serial calls cost latency; and a plausible intermediate answer can conceal missing evidence.

PRA relation and experiment. PRA may represent a follow-up target as a generated reference only when a resolver can bind it deterministically; free-form web search is not a URI lookup. Compare text subquestions, latent reference queries, and a hybrid that uses PRA for known namespaces and search otherwise. Use the same search backend and hop budget; measure decomposition accuracy, evidence recall, calls, latency, valid/shuffled reference effects, and failures where the correct source is absent from the advertised graph.

9.3 IRCOT

Motivation and mechanism. IRCOT interleaves retrieval with chain-of-thought so partial reasoning becomes a better query for the next evidence hop [Trivedi et al., 2023]. Rather than retrieve once from the original question,

$$z_t = R(q, r_{<t}, e_{<t}), \quad r_t = G(q, r_{<t}, e_{<t}, z_t), \quad (59)$$

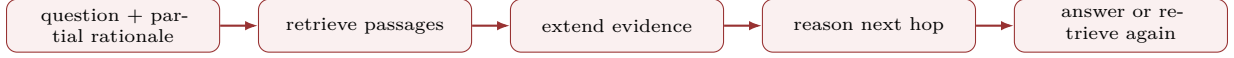
and the cycle continues until an answer or budget limit. Retrieval and reasoning therefore co-evolve.

Listing 41: IRCOT-style retrieval/reasoning alternation

```

state, evidence = question, []
for hop in range(max_hops):
    passages = retriever.search(state, k)
    evidence.extend(passages)
    state, answer = reasoner.step(question, state, passages)
    if answer is not None: return answer, evidence

```



Evidence, paper trail, and failures. IRCOT evaluates multi-step QA including HotpotQA, 2WikiMultiHopQA, MuSiQue, and related datasets, comparing retrieve-then-read, iterative retrieval, and chain-of-thought configurations with exact match/F1 and retrieval metrics. It motivates later iterative and agentic RAG. Reasoning text can introduce false entities that poison later queries, cost grows by hop, stopping is difficult, and answer gains can reflect a stronger prompt rather than a better retriever.

PRA relation and experiment. PRA layers or decoding states could emit successive latent queries, approximating IRCOT without serializing each query. Compare textual query rewriting and latent queries against the same frozen retriever, plus oracle evidence order. Match retrieval calls/candidates and report answer quality, hop recall, latency, and counterfactual evidence effects. The hybrid should expose an auditable object path even if queries are latent; otherwise reduced tokens come at the cost of provenance.

9.4 Toolformer

Motivation and mechanism. Toolformer teaches a language model when and how to call APIs by sampling candidate calls and retaining those that reduce language-model loss [Schick et al., 2023]. For call c inserted at position i , a simplified filter is

$$\Delta(c) = L(x_i: | x_{<i}) - L(x_i: | x_{<i}, c, \text{result}(c)); \quad c \text{ kept if } \Delta(c) > \tau. \quad (60)$$

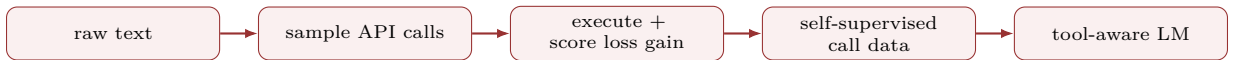
Fine-tuning on retained demonstrations yields discrete call triggering and result integration without dense human call labels.

Listing 42: Loss-improvement filtering for pseudo-labeled calls

```

for position in candidate_positions(text):
    for call in sample_calls(model, text, position):
        result = execute_sandboxed(call)
        gain = loss_without(call) - loss_with(call, result)
        if gain > threshold: training_examples.append(insert(text, call, result))

```



Evidence and limitations. Toolformer evaluates a 6.7B GPT-J model with calculator, QA, search, translation, and calendar tools on zero/few-shot downstream tasks and language modeling, comparing base and tool-aware models. Tool choice and API formatting ablations matter. Pseudo-labels inherit the base model’s blind spots, unsafe or costly calls require sandboxing, tool errors enter the context, and lower token loss does not guarantee factual or task utility.

PRA relation and experiment. Toolformer’s call-retention principle is directly useful for PRA router training: sample candidate object accesses, keep those whose controlled insertion lowers loss, then train a gate with corrupted negatives. Compare heuristic cosine, supervised relevance, and loss-filtered routers under equal trainable parameters. Evaluate selection recall, calibration, task loss, valid/shuffled/irrelevant effects, and access cost. PRA can internalize deterministic reads; arbitrary or side-effecting APIs should remain explicit tool calls.

9.5 WebGPT and browser agents

Motivation and mechanism. WebGPT trains a model to issue search queries, open results, navigate pages, collect quotations, and answer with citations, using demonstrations, behavior cloning, reward modeling, and human feedback [Nakano et al., 2021]. The browser state makes information acquisition a sequential decision process:

$$a_t \sim \pi_\theta(a | s_t), \quad s_{t+1} = \text{browser}(s_t, a_t), \quad y = G(q, \text{quotes}(\tau)). \quad (61)$$

Listing 43: Citation-preserving browser loop

```

state, evidence = browser.start(question), []
for _ in range(action_budget):
    action = policy(state, evidence)
    if action.kind == "quote": evidence.append(browser.quote(action.span))
    elif action.kind == "answer": return compose_with_citations(question, evidence)
    else: state = browser.execute(action)

```



Evidence, descendants, and failures. WebGPT evaluates long-form question answering using human preference, factuality, and citation-related judgments against retrieval and human-written baselines. It established a lineage toward browser-assisted research agents. Results are bound to the web snapshot, search engine, model, demonstrations, reward model, and evaluator. Failures include source-quality errors, selective quotation, citation drift, prompt injection, browsing loops, changing pages, and high serial latency.

PRA relation and experiment. PRA cannot discover an unknown open-web source. It can cache already discovered pages, versions, sections, and evidence for later questions. Compare fresh browser search, cached text RAG, cached PRA KVs, and a policy that searches on cache miss or staleness. Measure answer/citation correctness, source diversity, actions, latency, cache hit and avoided encoding, update detection, and security under malicious page instructions. Provenance must retain the original page/version even when KVs are reused.

9.6 Recursive Language Models and context as environment

Motivation and mechanism. Recursive language-model approaches treat an oversized context as an external environment. A model writes code or issues subcalls that inspect, partition, summarize, and recursively solve pieces instead of consuming the entire source in one forward pass. For context D and budget B ,

$$y = F(q, D, B) = \text{aggregate}\{F(q_j, D_j, B_j)\}_{j=1}^m, \quad \sum_j B_j \leq B. \quad (62)$$

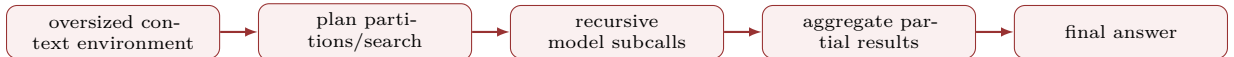
The model controls the decomposition and can adapt it to each question.

Listing 44: Budgeted recursive context program

```

def solve(query, view, budget):
    if view.fits() or budget.depth == 0:
        return model(query, view.read(budget.tokens))
    plan = model.plan_partitions(query, view.metadata(), budget)
    partials = [solve(p.subquery, view.slice(p.range), budget.child(p)) for p in plan]
    return model.aggregate(query, partials)

```

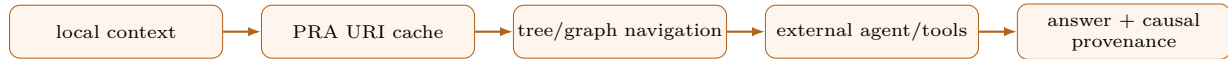


Evidence and failure modes. The family is evaluated on inputs beyond a model’s window, code/document analysis, aggregation, and long-context reasoning, with answer quality, subcall count, tokens, and cost. Baselines should include truncation, long context, RAG, and non-recursive map-reduce. Generality is high, but recursive calls are expensive, partition boundaries can destroy dependencies, summaries compound errors, arbitrary generated code needs sandboxing, and budgets/stopping rules can dominate outcomes.

PRA relation and experiment. PRA can be viewed as a differentiable cached fast path for recurring read operations discovered by an agent or recursive program. Use identical source partitions and compare RLM subcalls, tree navigation, PRA references, and a tiered system that escalates on uncertainty. Match total model FLOPs/tokens and wall-clock budget; report depth, calls, source coverage, provenance, cache reuse, and tasks whose relevant partition is unknown initially. The agent should win on open-ended discovery; PRA must justify itself through repeated low-latency reuse and causal content selection.

9.7 A tiered agent–PRA design

The categories are best composed as a monotone escalation policy. Local attention handles dense immediate context; a warm URI cache handles repeated named reads; a tree/graph index handles bounded structural search; and a general agent handles discovery or action. Each tier returns both evidence and calibrated confidence. Expensive discoveries may be cached only if the source is immutable or versioned, authorization permits reuse, and the action is read-only. The evaluation must include wrong-confidence costs: unnecessary escalation, silent cache hits on stale data, and premature stopping before an agent search.



10 Cross-Cutting Conditional Computation

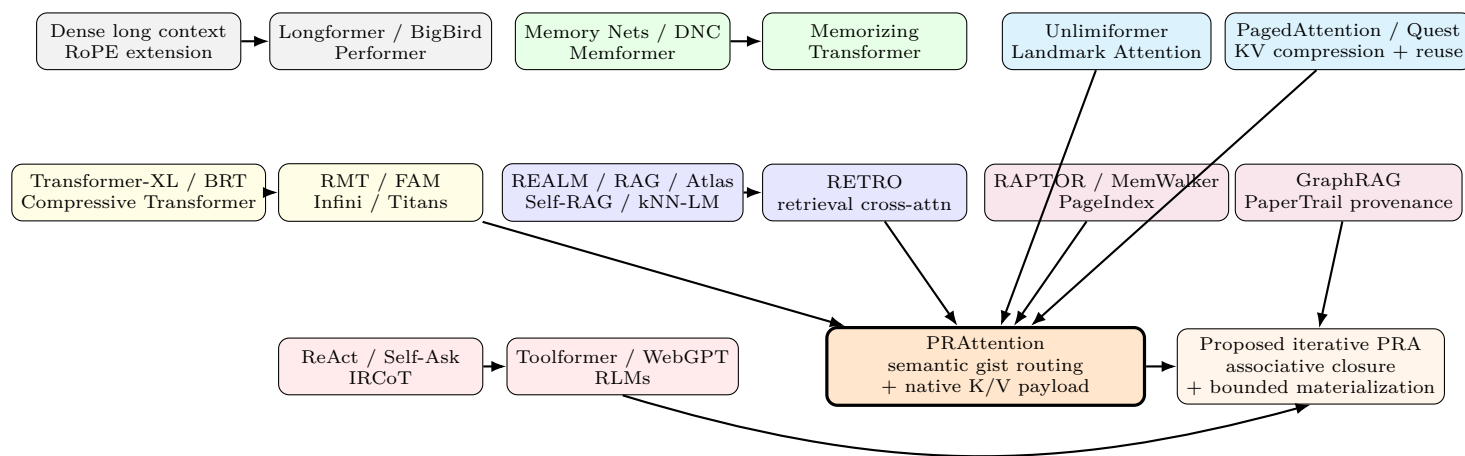
Memory selection is one instance of a wider question: how much computation should a model allocate to a token, example, or reasoning state? Adaptive Computation Time lets a recurrent model learn a halting distribution over internal steps [Graves, 2016]. Universal Transformers apply a shared self-attentive transition recurrently and can use adaptive halting per position [Dehghani et al., 2019]. These methods vary compute depth rather than memory content, but their accounting principle is the same. A learned controller must pay for its own scores, obey a hard or expected budget, and be compared with fixed computation at equal realized work.

PRA extends conditional allocation along three axes: which source is searched, how many retrieval hops run, and how much selected detail becomes active. A confidence threshold does not by itself bound any of them. Production-safe policies therefore need explicit maxima for candidate scoring, closure depth, unique sources, native-KV tokens, and enabled layers. Quality should be plotted against realized work, not only against a nominal threshold. This connection also clarifies terminology: progressive access is adaptive computation over memory, whereas recurrent depth is adaptive computation over representation refinement. The two may compose, but they answer different questions.

11 One-Page Architecture Landscape

The preceding method descriptions can obscure a simple pattern: architectures differ less by whether they “use memory” than by when they pay to expose information and what identity survives that exposure. The landscape below places the seven families on those common axes. Arrows indicate useful conceptual transitions, not a chronological claim or a relation of strict inheritance.

Landscape of Long-Context and Retrieval Architectures



Main axes: logical availability ↔ active computation; flat tokens ↔ named structure;
text ↔ latent/native K/V; one-shot selection ↔ associative closure; transient context ↔ reusable cache.

12 Cross-Architecture Comparison

The comparison treats each family as an access contract. Table 4 asks what exists and how it crosses the boundary. Table 5 asks when selection repeats, whether the host must be retrained, and whether active work is bounded independently of logical memory. Entries describe the canonical mechanism; individual implementations can compose several rows.

Table 4: Memory topology: stored content, search, compression, and materialization.

Family	Stored and where	Search unit	Compression	Materialized detail	Primary boundary
Dense long context	prompt / device K/V	none after prompt assembly	none or numeric precision	all token native K/V	sequence length and quadratic interaction
Sparse attention	prompt / device K/V	fixed or learned edge	graph sparsity	token native K/V on allowed edges	missing interaction path
Recurrent or SSM	chronological hidden state	temporal update	bounded latent state	recurrent vector/state	superposition and anonymous history
Test-time neural memory	online parameters/state	learned key or surprise	neural weights	latent read	contamination and correction
Differentiable memory	learned slots / model state	slot content	fixed slot count	latent vectors	address identity and write stability
Fixed RAG	external text corpus	vector/text chunk	embedding/index	retrieved text encoding	source/chunk miss
Iterative RAG	corpus plus partial reasoning	rewritten query	top- k per hop	text per hop	serial errors and stopping
RETRO	external token database	neighbor chunk	nearest-neighbor index	encoded neighbor states	corpus/index dependence
Landmark/latent retrieval	ingested hidden/KV blocks	landmark or latent block	gist/block index	selected hidden or native states	representative miss
KV eviction	active stream cache	importance or token rule	token removal	retained native K/V	irreversible omitted detail
KV paging/quantization	host/device backing K/V	page or none	layout / lower precision	loaded native K/V	bandwidth, bytes, quantization
Prompt/KV reuse	server cache	exact module/prefix	avoided recomputation	cached attention state	context or position mismatch
Tree/graph index	external summaries, nodes, edges	hierarchy/path	summaries or graph	text, spans, subgraph	construction and traversal error
Search agent	open sources/tools	generated action/query	observation filtering	tool observations/text	latency, action, provenance
One-shot PRA	URI/prompt-head objects and backing native K/V	semantic URI/chunk gist	compact routing index	selected full native K/V	route, depth, cold encoding
Iterative PRA (proposed)	same plus visited/path state	retrieved gist frontier	bounded beam/closure	final selected native K/V	hop miss, cycles, budget

Table 5: Execution policy. “Bounded” means active work can remain bounded while the logical store grows; it does not mean constant end-to-end latency.

Family	One-shot / iterative	Consumed form	Host training	Active bound	Question it best answers
Dense long context	one forward	native K/V	often context adaptation	no, grows with prompt	What if all evidence interacts?
Sparse attention	layerwise	native K/V	usually yes	pattern-dependent	Which token edges are allowed?
Recurrent/SSM	iterative over stream	latent state	yes	yes	What summary should chronology carry?
Neural memory	iterative reads/writes	latent/state parameters	yes	slot/state bound	What should be learned into mutable state?

Family	One-shot / iterative	Consumed form	Host training	Active bound	Question it best answers
Fixed RAG	one-shot or per segment	text/encoder states	retriever/reader varies	top- k	Which external passages should be inserted?
Iterative RAG/agent	iterative	text/observations	prompting or training	per-hop, not total unless capped	What new source becomes relevant next?
RETRO	per chunk	encoded neighbor states	yes	neighbors per chunk	Can nonparametric data scale pretraining?
Landmark/KV retrieval	per query/layer	hidden/native K/V	method- dependent	selected blocks/pages	Which latent block should be active?
KV management/reuse	runtime allocation	retained/reused native K/V	usually no	byte/page budget	How can serving retain or reuse states?
Tree/graph navigation	one-shot or iterative	summary, text, or graph	index construction	branch/depth budget	Which structural path exposes evidence?
One-shot PRA	per configured layer/step	selected native K/V	thin adapter/router	yes, K/V budget	Can named logical memory become a bounded working set?
Iterative PRA (proposed)	bounded hops	gist path then native K/V	expected router/controller	hop plus final K/V budgets	Can retrieved concepts close an evidence chain?

No row dominates every column. Dense context provides the broadest interaction graph after eager materialization. Recurrent and state-space systems bound active state by compressing chronology. RAG and structural indexes keep source text external but use different discovery and traversal rules. KV policies control bytes, bandwidth, and reuse without necessarily solving semantic discovery. Agents maximize discovery flexibility while adding serial model and tool calls. PRA is most directly compared with Landmark Attention, trained retrieval, Prompt Cache/CacheBlend, and hierarchical navigation. Its identity and native-KV transport matter only if they improve a measured quality–resource frontier after source encoding, routing, transfer, and integration are charged.

13 Adapting Original Experiments to PRA

13.1 A benchmark matrix

The repository example exposes several separable abilities: preserving ordinary language modeling, locating a named module, selecting the relevant method within it, and discovering an unadvertised source when the named objects are insufficient. A benchmark that tests only one ability cannot distinguish the access contracts above. A credible program therefore combines:

1. **Language preservation:** WikiText-2/103 or C4 slices before and after memory adaptation.
2. **Exact long-range retrieval:** passkeys, variable needles, UUID/value lookup, and repeated distractors.
3. **Natural long-document understanding:** NarrativeQA, Qasper, QuALITY, GovReport, arXiv/PubMed summarization, and LongBench-style tasks.
4. **Open-domain retrieval:** Natural Questions, TriviaQA, HotpotQA, 2WikiMultiHopQA, MuSiQue.
5. **Structured documents:** FinanceBench, legal filings, technical manuals, code repositories, and table/figure references.
6. **Agentic discovery:** web or corpus search tasks where the correct source is initially unknown.

The sequence is intentional. Language preservation is a compatibility gate. Synthetic retrieval isolates routing under known ground truth. Natural documents test whether the mechanism survives realistic discourse. Open-domain and agentic tasks should be added only when source discovery, rather than access to an advertised object, is itself under study.

13.2 Common conditions

For each task, the baseline set should include local-only SelfAttention, longer-context SelfAttention, sparse attention, recurrent or FAM state, one-shot RAG, Atlas-style trained retrieval, Self-RAG-style adaptive retrieval, Landmark or

latent token retrieval, PRA with explicit handles, structural-tree retrieval, and agentic search. Systems experiments should additionally compare full K/V, query-aware page selection, adaptive retention, CacheBlend-style reuse, and PRA object caches. PRA with learned source discovery is a separate system variant because it removes the advertised-object assumption.

Comparisons should hold the corpus, tokenizer, generation model, evaluation prompts, and random data splits fixed where meaningful. They should report both equal visible evidence budgets and equal end-to-end compute budgets. Matching only parameter count is insufficient when one method performs retrieval, encoding, or extra model calls outside the measured forward pass.

13.3 PRA-specific causal matrix

Task improvement does not by itself show that a reference mechanism selected or used the right evidence. The interventions below alter one part of the access chain at a time. For example, shuffled content preserves branch activation and approximate tensor shape while breaking the association between a handle and its evidence.

Condition	What it tests	Expected interpretation
Valid	complete system	task utility
Disabled branch	any memory contribution	Δ_{use}
Shuffled content	semantic dependence	$\Delta_{content}$
Irrelevant matched length	distractor resistance	router specificity
Random URI, same content	identity dependence	naming shortcut
Same URI, wrong version	cache invalidation	staleness safety
Oracle object	selection upper bound	router headroom
All evidence local	access overhead	efficiency frontier
Agent oracle search	discovery upper bound	unresolved-source headroom

The conditions are most informative when crossed rather than reported independently. If valid and shuffled references perform alike, the memory branch may be acting as an adapter rather than carrying content. If an oracle object helps but learned routing does not, representation and fusion remain plausible while selection is the bottleneck. If even the oracle fails, training or materialization should be repaired before scaling the router.

13.4 Metrics beyond task accuracy

Report task quality, retrieval recall, ranking, reference causality, preservation, and systems cost together. Selection metrics should use candidate-count-aware chance rates and distinguish reference recall from within-reference chunk, landmark-block, and KV-page recall. Triggering systems should report access precision and unnecessary retrievals. Causal metrics should report loss or answer changes under removal and substitution, not attention weight alone. Systems measurements must separate resolve time, encode time, cache lookup, routing, attention, generation, and synchronization in cold, warm, and shared-cache regimes. Cache hit rate without avoided work is not useful; latency reduction without answer preservation is not success.

14 Synthesis Across the Nine Questions

14.1 Availability and causal activity

Dense context makes availability and activity nearly coincide: after prefill, most prompt K/V remains available to attention. Sparse attention changes the interaction graph but usually retains the sequence. Recurrence and state-space models compress chronology into bounded state. RAG, structural indexes, agents, and PRA place a larger store beyond the active model operation. The relevant resource claim is therefore not “supports N tokens” but the relationship among logical source size, index size, active detail, and work per decision.

14.2 Discovery, geometry, and closure

Discovery ranges from none (a fixed prompt), through search over a known corpus, to structural navigation and open tools. Search geometry may be lexical, embedding-based, graph-based, attention-native, learned asymmetric, or generated as an action. PRA’s evidence shows why the choice is substantive: reproducing native Q/K rankings did not produce strong semantic ranking, whereas a small learned metric did. One-shot ranking still cannot discover a source whose relevance appears only through a retrieved concept. Iterative RAG, search agents, graph walks, and proposed associative-closure PRA address that separate problem.

14.3 Detail, depth, and behavior

Retrieved text asks the host to encode and integrate evidence in its familiar token pathway. Latent memories avoid repeated text but introduce representation compatibility. Native-KV transport preserves pretrained value content and attention head structure, yet its effect depends on layer placement and consumer calibration. The evaluation chain must continue from retrieval recall through oracle use, answer-critical logits, completion, and free-running behavior. QASPER demonstrates a positive one-shot integration regime; HotpotQA demonstrates that relation-near retrieval is not all-required-evidence closure.

14.4 Serving and resource frontiers

Paging, quantization, eviction, reuse, semantic selection, and materialization control different terms. They should be composed rather than credited interchangeably. An experiment must state whether it reduces FLOPs, active K/V tokens, HBM residency, host bytes, transfer, prefill, or repeated source encoding. Quality should be plotted against the resource actually changed, with cold and warm paths separated.

15 Research Lineage and Reusable Related-Work Synthesis

The field is best understood as several boundary shifts rather than one chronology. Memory Networks and differentiable computers separated storage and content addressing from recurrent state. Transformer-XL, recurrent memory models, selective state-space models, Infini-attention, and Titans made chronological state longer lived, more selective, or adaptive. REALM, RAG, Atlas, Self-RAG, and RETRO moved updateable knowledge outside parameters and changed when it is retrieved. Landmark Attention and latent K/V retrieval moved selection closer to attention. PagedAttention, Prompt Cache, CacheBlend, KIVI, and eviction policies separated allocation, reuse, precision, and bandwidth. RAPTOR, MemWalker, PageIndex, and GraphRAG restored hierarchy and relations. ReAct, IRCOT, Toolformer, WebGPT, and recursive context programs made discovery an explicit sequential policy. The following compact comparisons are intended to remain reusable in related-work sections:

Long context versus bounded memory. Long-context and sparse-attention methods enlarge or restructure the token interaction graph after evidence has been materialized. Bounded-memory methods keep a larger logical store outside the active graph and pay for selection. The former avoids retrieval misses; the latter can reduce active work when relevance is sparse. Matching nominal context length does not match their resource or failure surfaces.

RAG versus latent/native memory. RAG usually returns text or encoder states and lets the reader contextualize them for the current query. Latent and native-KV systems can avoid repeated text processing and enter closer to attention, but must preserve model, position, layer, and version compatibility. Retrieval quality and representation fidelity are independent experimental axes.

KV management versus semantic admission. Eviction, quantization, paging, and prefix reuse govern the physical lifecycle of states that have already been encoded. Semantic routing decides which source should become active. These mechanisms compose: a semantic miss cannot be repaired by better paging, and a good route can lose its systems advantage through wasteful transfer or storage.

One-shot retrieval versus associative closure. One-shot systems rank candidates from an initial query. Iterative retrievers, graph walkers, and agents allow retrieved concepts to formulate the next access. They should be compared at a fixed final unique-evidence budget and with path-level recall, because allocating more total candidates can imitate reasoning without improving selection.

PRA’s position. PRA combines named logical objects, compact semantic search, selected native-KV payload, and a bounded active working set. Each component has precedents; the contribution is their explicit separation and causal interface. Current evidence supports the one-shot native-KV form and depth-sensitive pretrained integration. Iterative closure, smart detail allocation, and production serving remain open.

16 Research Frontier

16.1 Multi-gist semantic resolution

Multiple subchunk means are a cheap way to reduce averaging loss, but measured increases in native-Q/K fidelity did not yield a monotone semantic-recall gain. Multi-gist work should therefore compare parameter-free segment means with learned semantic projections at equal index bytes and selected native-KV fraction. Failure of the parameter-free method strengthens the case for learned routing; success would provide a simpler and more interpretable index.

16.2 Iterative associative closure and path preservation

The next architectural question is whether retrieved gists can reveal another required object without rerunning the full transformer over every intermediate passage. The controller should retain root anchoring, visited identities, hop scores, and predecessor edges; enforce depth, beam, cycle, and final-unique-chunk budgets; and compare against one-shot top- B , IRCOT-style text iteration, and graph traversal. HotpotQA tests relational closure, while QASPER guards against regressing the established one-shot regime.

16.3 Progressive and smart materialization

Discovery returns relevant objects; materialization decides how much detailed native K/V each receives. Whole-chunk admission is the conservative baseline. Page bounds, progressive escalation, per-layer budgets, and token-level allocation may reduce active memory. The central factorial experiment crosses oracle versus learned object selection with full versus partial detail. It distinguishes a router miss from an allocator miss and tests the redundancy and attention-overload hypotheses described in the PRA chapter.

16.4 Production serving

The eventual systems claim requires indexed or approximate search, batched heterogeneous materialization, fused sparse attention, cache admission and eviction, quantized backing tiers, authorization, version invalidation, and observability. The target is an end-to-end frontier, not a microbenchmark: answer quality and causal evidence use versus cold/warm latency, throughput, active K/V, resident bytes, transfer, and avoided prefill under realistic sharing.

17 Cognition as a Bounded Analogy

The decomposition has cautious links to cognitive and economic models, but no architectural homology is implied. Working-memory research treats immediate processing capacity as limited and coordinated with longer-term knowledge [Baddeley, 1992]. Bounded rationality studies decisions under finite information and computational resources [Simon, 1955]. Rational-inattention models make information acquisition itself a costly allocation decision [Sims, 2003]. These traditions motivate useful questions for machine memory: which cues trigger access, what information becomes active, how interference changes decisions, and how effort adapts to expected value.

The analogy stops at those questions. Transformer K/V is not human episodic memory; a URI resolver is not biological recall; a learned top- k router is not a neural theory of attention; and a fixed token budget is not a psychometric capacity estimate. The machine mechanisms have explicit tensors, training objectives, security domains, and serving costs. Cognitive language is useful only when it yields a falsifiable computational intervention, such as cue corruption, interference, delayed access, or effort-quality trade-offs.

18 Discussion

The survey’s central conclusion is that long-context research is moving from a single context window toward a decomposition of memory computation. Logical availability, semantic discovery, detail transport, layer integration, decision formation, and serving are separable design problems. Architectures differ in where they compress information and where they risk an irreversible miss.

PRA makes this decomposition unusually explicit, but the evidence is deliberately narrower than the vision. The one-shot native-KV system demonstrates bounded active memory, positional and payload correctness, learned semantic routing, cross-family retrofit, and a positive QASPER adaptation regime. It also demonstrates unresolved router headroom, depth sensitivity, small evaluation cohorts, and a HotpotQA relational boundary. Those negative

boundaries are diagnostic: they motivate associative closure before indiscriminate consumer capacity, and they motivate detail allocation only after evidence identity is correct.

PRA should not be expected to dominate when sources are short and densely relevant, exact prefix reuse solves the workload, source discovery is open-ended, or a one-shot text retriever already fits the quality and latency target. Its plausible regime has large versioned objects, sparse query-specific relevance, repeated use, auditable identity, and enough workload reuse to amortize indexing and encoding. Even there, strong long-context, RAG, structural, KV-management, and agent baselines remain necessary.

19 Conclusion

An information source can be available without being active, relevant without containing all required detail, correctly routed without being integrated, integrated without changing the right decision, and decision-useful without producing a complete answer. A survey that treats all of these outcomes as “context use” cannot localize progress.

The nine-question framework supplies that localization across dense and sparse attention, recurrence and state-space models, differentiable and test-time memory, RAG, latent/KV retrieval, structural navigation, and agents. PRA is the worked synthesis: a measured one-shot architecture that separates semantic search from native execution and logical memory from an active K/V budget. Iterative associative closure is the next experiment; selective materialization and optimized serving are later layers of the research program. Keeping those statuses distinct makes the central claim both more modest and more useful: bounded memory computation is an emerging systems and modeling problem whose components can now be tested independently.

Online Resources

The PRA manuscripts, implementation, experiment artifacts, and build sources used in this survey are distributed together in the project repository. Live links for external works are embedded in the bibliography through DOI, proceedings, or arXiv URLs. Repository-only systems and all quantitative claims should be independently reproduced before external deployment or comparison.

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.11511>.
- Alan Baddeley. Working memory. *Science*, 255(5044):556–559, 1992. doi:10.1126/science.1736359.
- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2025. URL <https://arxiv.org/abs/2501.00663>.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. doi:10.18653/v1/2020.emnlp-main.263. URL <https://aclanthology.org/2020.emnlp-main.263/>.
- Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. Unlimiformer: Long-range transformers with unlimited length input. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.01625>.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, 2022. URL <https://proceedings.mlr.press/v162/borgeaud22a.html>.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling, 2024. URL <https://arxiv.org/abs/2406.02069>.

- Howard Chen, Ram Pasunuru, Jason Weston, Asli Celikyilmaz, Gheorghe Comanici, and Thibault Févry. MemWalker: Walking through long-context memory with an LLM, 2023a. URL <https://arxiv.org/abs/2310.05029>.
- Shouyuan Chen, Shengen Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation, 2023b. URL <https://arxiv.org/abs/2306.15595>.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2009.14794>.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. doi:10.18653/v1/P19-1285. URL <https://aclanthology.org/P19-1285/>.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1807.03819>.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. URL <https://arxiv.org/abs/2404.16130>.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs, 2023. URL <https://arxiv.org/abs/2310.01801>.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.
- Alex Graves. Adaptive computation time for recurrent neural networks, 2016. URL <https://arxiv.org/abs/1603.08983>.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. doi:10.1038/nature20101. URL <https://doi.org/10.1038/nature20101>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, 2020. URL <https://proceedings.mlr.press/v119/guu20a.html>.
- DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/d6e0bbb9fc3f4c10950052ec2359355c-Abstract-Conference.html.
- Dongseong Hwang, Weiran Wang, Zhuoyuan Huo, Khe Chai Sim, and Pedro Moreno Mengibar. TransformerFAM: Feedback attention is working memory, 2024. URL <https://arxiv.org/abs/2404.09173>.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models, 2022. URL <https://arxiv.org/abs/2208.03299>.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1911.00172>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi:10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2404.14469>.
- Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. MiniCache: KV cache compression in depth dimension for large language models, 2024a. URL <https://arxiv.org/abs/2405.14366>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024b. doi:10.1162/tacl_a_00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.17118>.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, 2024c. URL <https://arxiv.org/abs/2402.02750>.
- Anna Martin-Boyle, Cara A. C. Leckey, Martha C. Brown, and Harmanpreet Kaur. PaperTrail: A claim-evidence interface for grounding provenance in LLM-based scholarly Q&A. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 2026. URL <https://arxiv.org/abs/2602.21045>.
- Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers, 2023. URL <https://arxiv.org/abs/2305.16300>.
- Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention, 2024. URL <https://arxiv.org/abs/2404.07143>.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. WebGPT: Browser-assisted question-answering with human feedback, 2021. URL <https://arxiv.org/abs/2112.09332>.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2022. URL <https://arxiv.org/abs/2210.03350>.
- Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1911.05507>.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2401.18059>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Jorge Simão. Progressive retrieval attention: A position paper on demand-driven context expansion. Working position paper, 2026a.
- Jorge Simão. Progressive retrieval attention for pretrained transformers: Sparse native-K/V memory with semantic routing. Working paper and accompanying Hugging Face integration, 2026b.
- Jorge Simão. Position, attention geometry, and semantic routing for retrieved native-KV memory. Working paper and accompanying experiment artifacts, 2026c.
- Jorge Simão. Progressive retrieval attention: Bounded sparse native-KV for addressable logical memory. Working paper and accompanying open-source implementation, 2026d.
- Herbert A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955. doi:10.2307/1884852.

- Christopher A. Sims. Implications of rational inattention. *Journal of Monetary Economics*, 50(3):665–690, 2003. doi:10.1016/S0304-3932(03)00029-1.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021. URL <https://arxiv.org/abs/2104.09864>.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, pages 47901–47911, 2024. URL <https://proceedings.mlr.press/v235/tang24l.html>.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. doi:10.18653/v1/2023.acl-long.557. URL <https://aclanthology.org/2023.acl-long.557/>.
- Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1410.3916>.
- Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. Memformer: A memory-augmented transformer for sequence modeling, 2020. URL <https://arxiv.org/abs/2010.06891>.
- Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2203.08913>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. URL <https://arxiv.org/abs/2309.17453>.
- Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of NAACL-HLT*, 2016. doi:10.18653/v1/N16-1174. URL <https://aclanthology.org/N16-1174/>.
- Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion, 2024. URL <https://arxiv.org/abs/2405.16444>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2007.14062>.
- Mingtian Zhang, Yu Tang, and PageIndex Team. PageIndex: Next-generation vectorless, reasoning-based RAG. PageIndex blog and open-source repository, 2025. URL <https://github.com/VectifyAI/PageIndex/commit/7592163e2a376b3917181fff9ac1858dc5daa2c6>. Repository commit 7592163e2a376b3917181fff9ac1858dc5daa2c6; accessed 3 August 2026.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O : Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.14048>.